ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

Projet de Fin d'Année (PFA)

WC Fantasy

3ème Année Ingénierie Informatique et Réseaux

Réalisé par :

- BOURIZK AYOUB
- BEN MOUSA KARIM
- RAIH DOUAE
- SIMILLA HIDAYA
- EL JARJAF HAITAM AMINE

Membres du Jury :

Pr. EL KAYSOUNI Wafae	Qualité (Encadrante Pédagogique)
Pr. ABROUN Soundouss	Qualité (Examinatrice)

Année Académique : **2025 - 2026**

Remerciements

Nous tenons à exprimer notre sincère gratitude à notre encadrante pédagogique, **Pr. EL KAYSOUNI Wafae**, pour ses précieux conseils, son soutien constant et la qualité de son encadrement tout au long de ce projet. Sa disponibilité et son expertise ont été déterminantes dans l'aboutissement de ce travail.

Nous remercions également l'ensemble du corps enseignant de l'EMSI et plus particulièrement le département Informatique et Réseaux pour la qualité de la formation dispensée et les compétences techniques transmises durant ces trois années de cursus.

Enfin, nous adressons nos remerciements aux membres de notre groupe pour leur engagement, leur sérieux et la qualité de la collaboration maintenue tout au long du projet.

Résumé

Le présent rapport décrit la conception et le développement de **WC Fantasy**, une application mobile de fantasy football dédiée à la Coupe du Monde 2026. Face au manque d'engagement communautaire et à l'absence de plateformes optimisées pour les compétitions internationales de courte durée, ce projet propose une solution innovante permettant aux fans de football de s'immerger pleinement dans l'événement. En offrant la possibilité de constituer des équipes virtuelles avec des joueurs réels, de s'affronter au sein de ligues privées entre amis, et de participer à des batailles hebdomadaires gamifiées, WC Fantasy transforme les simples spectateurs en acteurs engagés de la compétition. L'intégration de fonctionnalités sociales riches, telles qu'un chatbot IA d'assistance et des systèmes de prédictions communautaires, contribue à créer une expérience utilisateur moderne, fluide et captivante, répondant ainsi parfaitement aux attentes d'un public passionné lors du plus grand événement sportif mondial.

Mots-clés : Fantasy Football, Gamification, Engagement Communautaire, Application Mobile, Coupe du Monde 2026

Abstract

This report presents the design and development of **WC Fantasy**, a mobile fantasy football application dedicated to the 2026 FIFA World Cup. Addressing the lack of community engagement and the absence of platforms optimized for short-term international tournaments, this project offers an innovative solution that allows football fans to fully immerse themselves in the event. By enabling users to build virtual teams with real-world players, compete against friends in private leagues, and participate in gamified weekly battles, WC Fantasy transforms passive spectators into active, engaged participants. The integration of rich social features, including an AI-powered assistant chatbot and community prediction systems, creates a modern, seamless, and captivating user experience, perfectly meeting the expectations of a passionate audience during the world's largest sporting event.

Keywords : Fantasy Football, Gamification, Community Engagement, Mobile Application, FIFA World Cup 2026

Table des matières

Remerciements I

Résumé II

Abstract III

Liste des figures VII

Liste des tableaux IX

Introduction générale 1

1 Contexte et Problématique 2

1 Contexte du projet 3

2 Problématique 3

3 Objectifs du projet 4

2 État de l'Art 5

1 Introduction 6

2 Présentation des travaux existants 6

2.1 Fantasy Premier League (FPL) 6

2.2 Sorare 6

2.3 FotMob (Mode Fantasy) 7

3 Tableau comparatif des travaux 7

4 Synthèse critique et positionnement 8

3 Spécification des Besoins et Conception du Système 10

1 Analyse des besoins 11

1.1 Besoins fonctionnels 11

1.2 Besoins non fonctionnels 11

2 Identification des acteurs 12

3 Cahier des charges synthétique 12

4 Conception générale — Architecture globale 13

5 Modélisation fonctionnelle — Diagramme de cas d'utilisation 14

6 Modélisation structurelle — Schéma de base de données 17

7 Modélisation structurelle — Diagramme de classes 19

8 Modélisation dynamique — Diagrammes d'activité 21

8.1 Diagramme 1 : Authentification utilisateur 21

8.2 Diagramme 2 : Composition de l'équipe (Draft) 22

8.3 Diagramme 3 : Prédictions communautaires 23

8.4 Diagramme 4 : Résolution des batailles 24

8.5 Diagramme de Séquence : Synchronisation des Scores 25

9 Choix technologiques 26

4 Développement, Validation et Résultats 29

1 Environnement de développement 30

2 Implémentation du système 30

2.1 Structure du projet backend 31

2.2 Authentification et sécurité 32

2.3 Gestion de l'équipe et des transferts 32

2.4 Calcul des points fantasy 33

2.5 Temps réel avec Socket.io 33

2.6 Synchronisation automatique des scores 33

2.7 Chatbot IA — Google Gemini 34

3 Interfaces utilisateur 34

3.1 Interfaces Mobiles 34
3.2 Interfaces d'Administration 37
4 Validation et tests 38
5 Difficultés rencontrées et solutions apportées 40
6 Limites du système 40

Conclusion générale **42**

Bibliographie et Webographie **44**

Annexes **46**

Annexe A — Endpoints REST principaux 47
Annexe B — Variables d'environnement (.env) 48
Annexe C — Barème complet des migrations SQL 48

Table des figures

2.1 Positionnement de WC Fantasy face aux solutions existantes 9

3.1 Architecture globale du système WC Fantasy 13
3.2 Diagramme de cas d'utilisation — Utilisateur 15
3.3 Diagramme de cas d'utilisation — Administrateur 16
3.4 Modèle Conceptuel de Données (MCD) 18
3.5 Diagramme de classes du système WC Fantasy 20
3.6 Diagramme d'activité — Authentification utilisateur 22
3.7 Diagramme d'activité — Composition de l'équipe (Draft) 23
3.8 Diagramme d'activité — Prédictions communautaires 24
3.9 Diagramme d'activité — Résolution des batailles 25
3.10 Diagramme de séquence — Processus de synchronisation des scores 26

4.1 Architecture MVC du backend WC Fantasy 31
4.2 Écran d'authentification 34
4.3 Écran profil utilisateur 34
4.4 Page d'accueil de l'application 35
4.5 Actualités et nouveautés 35
4.6 Composition de l'équipe (Draft) 36
4.7 Détails et statistiques d'un joueur 36
4.8 Suivi des matchs 37
4.9 Chatbot IA (Gemini) 37
4.10 Interface d'administration — Saisie des scores 38
4.11 Interface d'administration — Gestion des actualités 3

Liste des tableaux

2.1 Tableau comparatif des solutions existantes 8

3.1 Besoins fonctionnels du système 11
3.2 Exigences non fonctionnelles du système 12
3.3 Récapitulatif des choix technologiques 13
3.4 Schéma de la base de données 17
3.5 Justification des choix technologiques 27

4.1 Environnement de développement et pile technologique 30
4.2 Structure du projet 31
4.3 Barème de points fantasy 33
4.4 Scénarios de tests réalisés 39
4.5 Difficultés rencontrées et solutions apportées 40
4.6 Principaux endpoints REST de l'API 47
4.7 Variables d'environnement 48
4.8 Liste des migrations SQL 49

Introduction générale

Le football est sans conteste le sport le plus suivi au monde, réunissant des milliards de passionnés à chaque grande compétition internationale. La Coupe du Monde 2026, organisée conjointement par le Maroc, les États-Unis et le Canada, s'annonce comme l'édition la plus grande de l'histoire avec 48 équipes participantes, offrant ainsi un contexte exceptionnel pour les applications de divertissement sportif numérique.

Dans ce contexte, les applications de fantasy sport connaissent une croissance exponentielle. Ces plateformes permettent aux fans de participer activement à leur passion en constituant des équipes virtuelles à partir de joueurs réels, dont les performances sont ensuite converties en points. Le marché mondial du fantasy sport était estimé à plus de 20 milliards de dollars en 2023 et devrait dépasser 48 milliards d'ici 2030.

Problématique : La majorité des plateformes de fantasy sport existantes (Fantasy Premier League, FotMob, Sorare) sont conçues pour les championnats nationaux réguliers et ne proposent pas d'expérience optimisée pour une compétition éphémère comme la Coupe du Monde. De plus, elles manquent souvent d'interactions sociales riches (batailles directes, chat en temps réel, prédictions communautaires) et d'une interface mobile moderne et fluide.

Ce projet a pour objectif général de concevoir et développer **WC Fantasy**, une application mobile complète dédiée à la Coupe du Monde 2026, permettant à des groupes d'amis de s'affronter dans des ligues privées ou publiques via un système de fantasy football complet et gamifié. Les objectifs spécifiques sont : (1) développer un backend sécurisé et performant avec Node.js/Express/PostgreSQL ; (2) concevoir une application mobile cross-platform avec Flutter ; (3) implémenter des fonctionnalités temps réel via Socket.io ; (4) intégrer un module de synchronisation automatique des scores via une API officielle.

La méthodologie adoptée est itérative : analyse des besoins, conception de l'architecture, développement par fonctionnalités, et validation continue. Le présent rapport est organisé en quatre chapitres : le Chapitre 1 présente le contexte et la problématique ; le Chapitre 2 analyse l'état de l'art ; le Chapitre 3 expose la conception du système ; le Chapitre 4 détaille le développement et les résultats obtenus.

Chapitre 1

Contexte et Problématique

Introduction

Ce chapitre présente le contexte général dans lequel s'inscrit le projet WC Fantasy, définit la problématique à résoudre et précise les objectifs poursuivis. Il constitue le fondement sur lequel repose l'ensemble de la démarche de conception et de développement.

1. Contexte du projet

Le football est le sport le plus populaire au monde avec **plus de 5 milliards de fans** selon la FIFA [13]. La Coupe du Monde 2026 représente un événement sans précédent : pour la première fois dans l'histoire de la compétition, **48 nations s'affronteront**, générant **104 matchs** sur un mois et demi. Cet événement crée une opportunité unique pour des applications de divertissement sportif.

Le fantasy sport est un secteur en plein essor qui permet aux supporters de s'impliquer activement dans leur passion. Le principe est simple : chaque utilisateur constitue une **équipe virtuelle** composée de joueurs réels, et les performances réelles de ces joueurs (buts, passes décisives, clean sheets, etc.) se traduisent en points dans le jeu [12]. Ces points déterminent un **classement** au sein de ligues privées ou publiques.

En Afrique du Nord et au Maroc en particulier, l'intérêt pour le football est exacerbé par la **première participation historique du Maroc à la co-organisation** d'une Coupe du Monde, créant un terreau extrêmement favorable pour ce type d'application. L'essor de la téléphonie mobile (**taux de pénétration supérieur à 80% au Maroc**) renforce cette opportunité.

2. Problématique

Malgré la popularité du fantasy sport, plusieurs problèmes persistent dans les solutions actuellement disponibles :

- **Absence d'application dédiée** à la Coupe du Monde 2026 avec un catalogue complet de joueurs des 48 nations participantes.
- Les plateformes existantes sont **peu adaptées** aux compétitions éphémères (durée limitée, format à élimination directe).
- **Manque de fonctionnalités sociales** : chat en temps réel entre membres d'une ligue, batailles directes (head-to-head), prédictions communautaires.
- **Interfaces mobiles souvent non optimisées**, lentes et peu gamifiées.
- **Absence d'intelligence artificielle** pour enrichir l'expérience utilisateur (recommandations, chatbot d'assistance).

La question centrale est donc : ***comment concevoir une application mobile de fantasy football complète, performante et engageante, spécifiquement adaptée au format et à l'échelle de la Coupe du Monde 2026 ?***

3. Objectifs du projet

Objectif général : Concevoir et développer une application mobile de fantasy football dédiée à la Coupe du Monde 2026, offrant une expérience utilisateur complète, sécurisée et en temps réel.

Objectifs spécifiques :

1. **Offrir une expérience utilisateur immersive** avec une interface moderne, réactive et facile d'utilisation pour tous les profils de supporters.
2. **Créer un environnement hautement social** permettant aux utilisateurs de créer des ligues privées, de communiquer en temps réel et de s'affronter directement (Head-to-Head).
3. **Assurer un suivi en temps réel** des performances des joueurs lors de la compétition, avec un calcul de points instantané.
4. **S'adapter au format court d'une Coupe du Monde** avec une dynamique de jeu nerveuse, des prédictions de matchs et des récompenses régulières.

Conclusion

Ce chapitre a permis de situer le projet WC Fantasy dans son contexte technique, social et économique. La problématique clairement identifiée — absence d'une plateforme mobile dédiée à la Coupe du Monde 2026 avec des fonctionnalités sociales riches — justifie pleinement la conception d'une solution sur mesure. Le chapitre suivant présente une étude commerciale du marché et analyse les solutions existantes (leurs forces, faiblesses et les retours des utilisateurs) afin de définir le positionnement de notre projet.

Chapitre 2

État de l’Art

1. Introduction

Ce chapitre présente une synthèse critique des solutions et plateformes existantes dans le domaine des jeux de *Fantasy Sport*. Il est essentiel de passer en revue cette littérature et ces systèmes existants afin de comprendre les approches actuelles, d'identifier ce qui fonctionne et, surtout, de mettre en évidence les lacunes qui subsistent. Nous couvrirons les axes suivants : l'analyse détaillée des leaders du marché (Fantasy Premier League, Sorare, FotMob), la comparaison de leurs caractéristiques via un tableau de synthèse, et enfin notre synthèse critique qui justifiera le positionnement technique et fonctionnel de notre projet **WC Fantasy**.

2. Présentation des travaux existants

Dans le cadre de ce projet d'application, les « travaux existants » correspondent aux plateformes majeures dominant actuellement le marché du Fantasy Football. Pour chaque solution, nous analyserons son contexte, son approche technique, ses résultats et ses limites.

2.1. Fantasy Premier League (FPL)

a. Contexte et objectif : La FPL est le jeu officiel de la Premier League anglaise. Son objectif est de fidéliser les fans tout au long d'une saison de 38 journées en leur permettant de gérer une équipe virtuelle avec un budget défini.

b. Approche et technologies utilisées : La plateforme utilise une architecture web et mobile classique basée sur un système de *salary cap* (plafond salarial). Les points sont calculés de manière centralisée en fonction d'actions réelles sur le terrain (buts, passes décisives, *clean sheets*).

c. Résultats obtenus : La FPL a accompli l'exploit de rassembler une immense communauté mondiale (plus de 10 millions de joueurs actifs). Ses points forts résident dans la robustesse de son système de notation et l'équilibre de son économie virtuelle.

d. Limites identifiées : Ce travail présente une lacune sociale majeure : l'absence d'outils de communication intégrés, obligeant les utilisateurs à utiliser des applications tierces (WhatsApp, Discord) pour échanger au sein de leur ligue. De plus, son système est très rigide et ne s'adapte pas aux formats courts d'une Coupe du Monde.

2.2. Sorare

a. Contexte et objectif : Sorare est une plateforme innovante qui cherche à moderniser le Fantasy Sport en y intégrant la collection numérique de cartes de joueurs.

b. Approche et technologies utilisées : L'application repose sur la technologie Blockchain (Ethereum) et les NFTs (Non-Fungible Tokens). L'architecture permet aux utilisateurs d'acheter, vendre et échanger des cartes numériques dont la rareté (limitée, rare, super rare, unique) est cryptographiquement garantie.

c. Résultats obtenus : Sorare a révolutionné le marché avec son modèle économique novateur. Ses points forts sont un design premium, une forte composante d'innovation, et une excellente couverture de nombreux championnats internationaux.

d. Limites identifiées : La principale faiblesse de cette plateforme est son modèle *"Pay-to-Win"*. Les joueurs occasionnels qui ne souhaitent pas dépenser d'argent réel se retrouvent fortement désavantagés dans les classements, créant une barrière à l'entrée très élevée.

2.3. FotMob (Mode Fantasy)

a. Contexte et objectif : Initialement une application leader pour le suivi des scores en direct, FotMob a développé un mini-jeu de fantasy intégré pour accroître la rétention et l'engagement de ses utilisateurs pendant les matchs.

b. Approche et technologies utilisées : C'est une approche "mobile-first" s'appuyant sur l'intégration de flux de données en temps réel ultra-rapides. Le mode fantasy est greffé sur l'architecture existante de suivi des statistiques des matchs.

c. Résultats obtenus : FotMob offre une interface utilisateur exceptionnelle et des données instantanées extrêmement fiables. Le jeu est très fluide, rapide à prendre en main et très accessible pour les néophytes.

d. Limites identifiées : Le mode fantasy ne couvre pas les besoins compétitifs approfondis et est souvent perçu comme un simple gadget. L'application manque de ligues privées avancées et ne permet pas d'affrontements directs en duel (Head-to-Head) entre amis.

3. Tableau comparatif des travaux

Le tableau ci-dessous synthétise les similitudes et différences entre les travaux étudiés.

Référence	Technologie / Approche	Fonctionnalités couvertes	Points forts	Limites
FPL	Web/Mobile MVC, *Salary cap*	Composition (Draft), Ligues privées, Budget	Communauté massive, barème équilibré	Manque de social (pas de chat), inadapté aux tournois courts
Sorare	Blockchain, Web3, Cartes NFT	Collection, Achat/Vente, Tournois globaux	Innovation technologique, Design premium	Modèle *Pay-to-Win*, barrière financière élevée
FotMob	Intégration data temps réel	Scores en direct, Mode fantasy basique	Fiabilité des données instantanées, Fluidité UX	Pas de ligues avancées, perçu comme un gadget

TABLE 2.1 – Tableau comparatif des solutions existantes

Ce tableau compare les trois principales solutions existantes selon leur approche technologique, les fonctionnalités couvertes, leurs points forts et leurs limites, permettant de positionner clairement WC Fantasy parmi elles.

4. Synthèse critique et positionnement

L'analyse des travaux existants permet de dégager plusieurs constats majeurs.

Tout d'abord, on observe des **tendances communes** fortes : la gamification du suivi des matchs est devenue indispensable pour capter l'attention des fans modernes. Toutes les plateformes s'efforcent d'offrir des interfaces léchées et s'appuient massivement sur des flux de données statistiques en temps réel.

Cependant, nous avons pointé des **lacunes récurrentes** que ces solutions n'ont pas encore bien adressées. D'une part, l'aspect social est paradoxalement négligé (FPL, FotMob) : l'absence d'outils de communication intégrés brise l'immersion des joueurs. D'autre part, les modèles économiques modernes (Sorare) tendent à exclure le public occasionnel via des mécaniques de monétisation intrusives (*Pay-to-Win*). Enfin, la grande majorité de ces mécaniques sont pensées pour des saisons régulières longues et se révèlent inadaptées au format condensé d'un tournoi comme la Coupe du Monde.

Positionnement de WC Fantasy : Notre projet comble ces lacunes de manière ciblée.

Pour contrer le manque d'interaction, WC Fantasy intègre nativement un chat en temps réel (via Socket.io) au sein de chaque ligue. Pour corriger le déséquilibre financier, nous proposons un modèle équitable axé uniquement sur l'expertise footballistique. Enfin, pour nous adapter au rythme effréné de la Coupe du Monde 2026, nous introduisons un système de batailles directes (*Head-to-Head*) hebdomadaires, garantissant un enjeu continu.

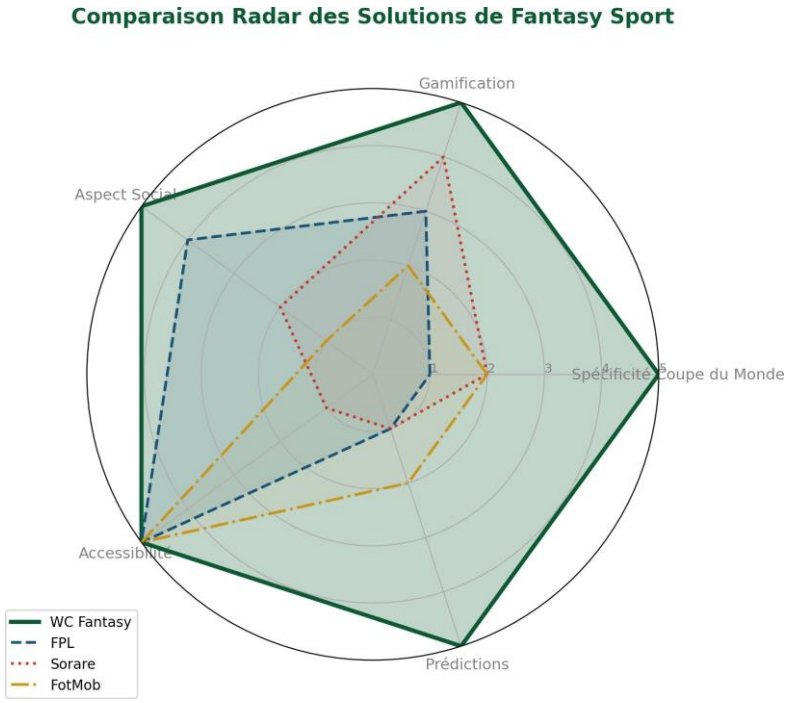


FIGURE 2.1 – Positionnement de WC Fantasy face aux solutions existantes

Ce diagramme radar illustre comment WC Fantasy se démarque en maximisant les axes liés au social, à la gamification et à l'adaptation aux tournois courts.

Conclusion

Ce chapitre a permis de dresser un état de l'art critique des plateformes de Fantasy Sport actuelles. Nous avons mis en évidence que malgré des offres techniquement abouties, les besoins d'intégration sociale et d'équité pour des événements courts restent non satisfaits. Ces conclusions justifient la pertinence et l'existence de **WC Fantasy**. Le chapitre suivant s'appuiera sur ce constat pour détailler la spécification des besoins et la conception technique du système conçu pour relever ces défis.

Chapitre 3

Spécification des Besoins et Conception du Système

Introduction

Ce chapitre formalise les besoins du système WC Fantasy, identifie les acteurs, définit le cahier des charges et présente la conception complète : architecture globale, diagrammes UML (cas d'utilisation, activité), schéma de base de données et choix technologiques justifiés.

1. Analyse des besoins

1.1. Besoins fonctionnels

#	Besoin fonctionnel
BF01	S'inscrire, se connecter et gérer son profil utilisateur
BF02	Consulter le catalogue complet des joueurs de la CdM 2026 avec stats et photos
BF03	Composer une équipe de 15 joueurs dans un budget de 100M avec contraintes de positions
BF04	Créer ou rejoindre une ligue privée ou publique via code d'invitation
BF05	Effectuer des transferts de joueurs (entrant/sortant) en période de transfert
BF06	Participer à des batailles hebdomadaires head-to-head contre d'autres équipes de la ligue
BF07	Consulter les scores et classements de la ligue en temps réel
BF08	Soumettre des prédictions communautaires avant chaque journée (top buteur, buts, nation)
BF09	Communiquer avec les membres de la ligue via un chat temps réel
BF10	Interagir avec un chatbot IA pour obtenir des informations et conseils
BF11	Gérer les actualités football (admin) et les consulter (utilisateur)
BF12	Compléter des défis quotidiens/hebdomadaires pour gagner des points XP
BF13	Réinitialiser son mot de passe par email (lien sécurisé, TTL 1h)
BF14	Administrer : joueurs, scores de GW, utilisateurs, prédictions (rôle admin)

TABLE 3.1 – Besoins fonctionnels du système

Ce tableau énumère les besoins fonctionnels (BF) requis pour le bon fonctionnement de l'application, couvrant les fonctionnalités utilisateurs et administratives.

1.2. Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse API < 200 ms pour 95% des requêtes grâce au cache Redis
Scalabilité	Architecture stateless permettant la multiplication des instances backend
Sécurité	JWT avec rotation de refresh token, hachage bcrypt (10 rounds), rate limiting
Disponibilité	Reconnexion automatique Socket.io, retry interceptor côté Flutter
Fiabilité	Transactions PostgreSQL ACID pour toutes les opérations financières (budget)
Maintenabilité	Architecture MVC côté backend, code modulaire et centralisé (points-Calculator)
Compatibilité	Application Flutter compatible Android 5.0+ et iOS 12+
Accessibilité	Interface intuitive, temps d'apprentissage < 5 minutes pour un nouvel utilisateur

TABLE 3.2 – Exigences non fonctionnelles du système

Ce tableau récapitule les exigences non fonctionnelles de sécurité, de performance, de scalabilité et de compatibilité nécessaires pour garantir la qualité de service de la plateforme.

2. Identification des acteurs

Le système WC Fantasy implique deux acteurs principaux :

- **Utilisateur (User)** : tout membre inscrit sur la plateforme. Il peut composer son équipe, rejoindre/créer des ligues, effectuer des transferts, participer aux batailles, prédire, chatter et interagir avec le chatbot IA.
- **Administrateur (Admin)** : utilisateur disposant de droits étendus. Il gère les joueurs du catalogue, saisit les scores réels après chaque journée de jeu, publie des actualités, résout les prédictions communautaires et administre les utilisateurs.

3. Cahier des charges synthétique

Ce tableau récapitule les choix technologiques de l'architecture globale en listant les outils utilisés pour le backend, l'application mobile, le temps réel et l'administration.

Récapitulatif des choix technologiques	
Backend	Node.js + Express.js — API REST[1] — PostgreSQL — Redis — Socket.io
Mobile	Flutter (Dart) — Riverpod (gestion état) — Dio (HTTP) — Flutter-SecureStorage
Auth	JWT (access 7j + refresh 30j) — bcrypt — rate limiting — CORS configurable
Temps réel	Socket.io — rooms par ligue et par bataille — rate limit 1 msg/sec
Scoring	Barème détaillé par position — synchronisation auto via football-data.org
Gamification	XP, défis quotidiens/hebdomadaires, achievements débloquables
IA	Chatbot Google Gemini — module prédictions communautaires
Email	Brevo SMTP — reset mot de passe — email de bienvenue

TABLE 3.3 – Récapitulatif des choix technologiques

4. Conception générale — Architecture globale

L'architecture du système repose sur un modèle client-serveur découplé en trois couches principales. Le schéma ci-dessous illustre l'organisation générale du système et les interactions entre ses composants :

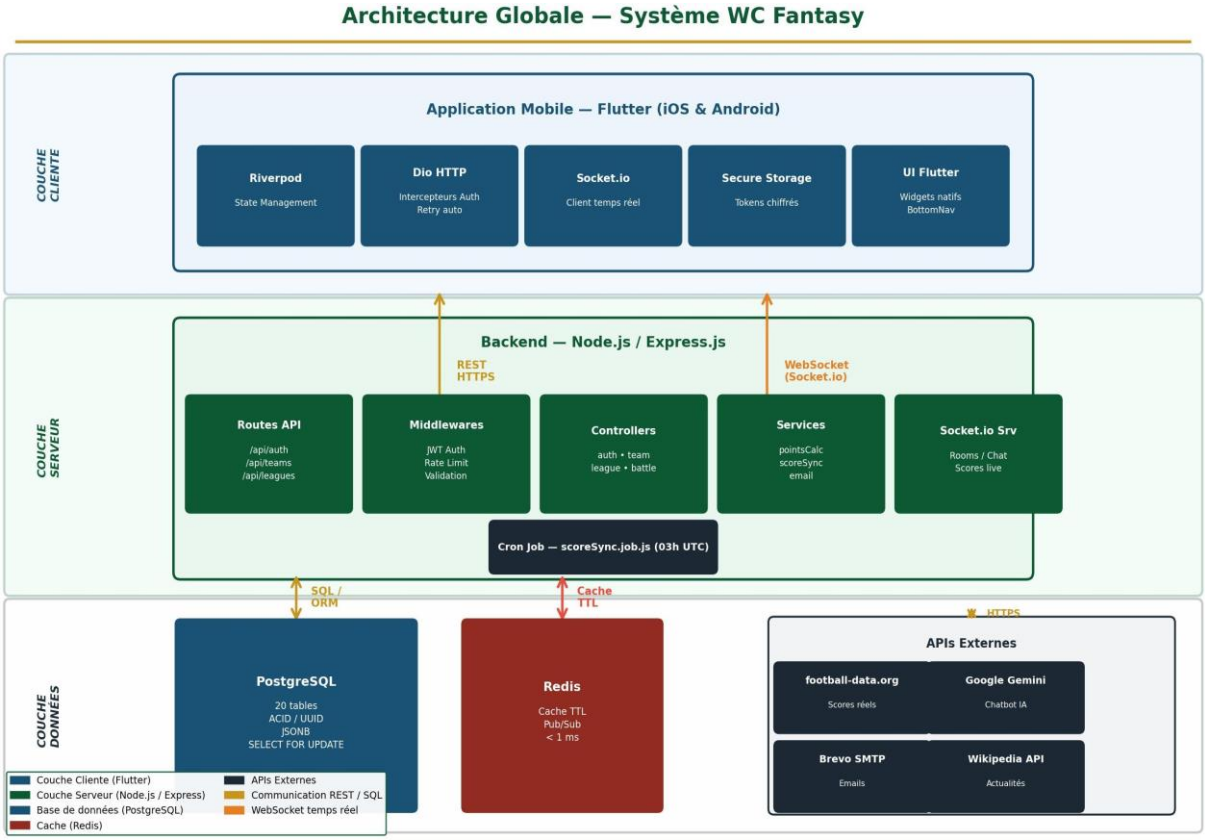


FIGURE 3.1 – Architecture globale du système WC Fantasy

Cette figure schématise l'interaction entre l'application mobile (Flutter), le serveur backend (Node.js/Express) et les différentes bases de données. Elle montre comment les requêtes HTTP et les événements WebSocket transitent pour fournir une expérience temps réel à l'utilisateur.

L'architecture se décompose en trois couches distinctes :

- **Couche Cliente (Flutter)** : interface utilisateur cross-platform, gestion de l'état avec Riverpod, communication HTTP via Dio avec intercepteurs d'authentification et retry automatique.
- **Couche Serveur (Node.js/Express)** : logique métier organisée en Controllers, Routes et Middlewares, serveur Socket.io intégré, tâche cron de synchronisation des scores, service email.
- **Couche Données** : PostgreSQL pour la persistance (20 tables), Redis pour le cache (TTL configurable), APIs externes (football-data.org, Google Gemini, Brevo SMTP, Wikipedia).

5. Modélisation fonctionnelle — Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous recense l'ensemble des interactions entre les deux acteurs (Utilisateur et Administrateur) et le système. Il est organisé par domaine fonctionnel.

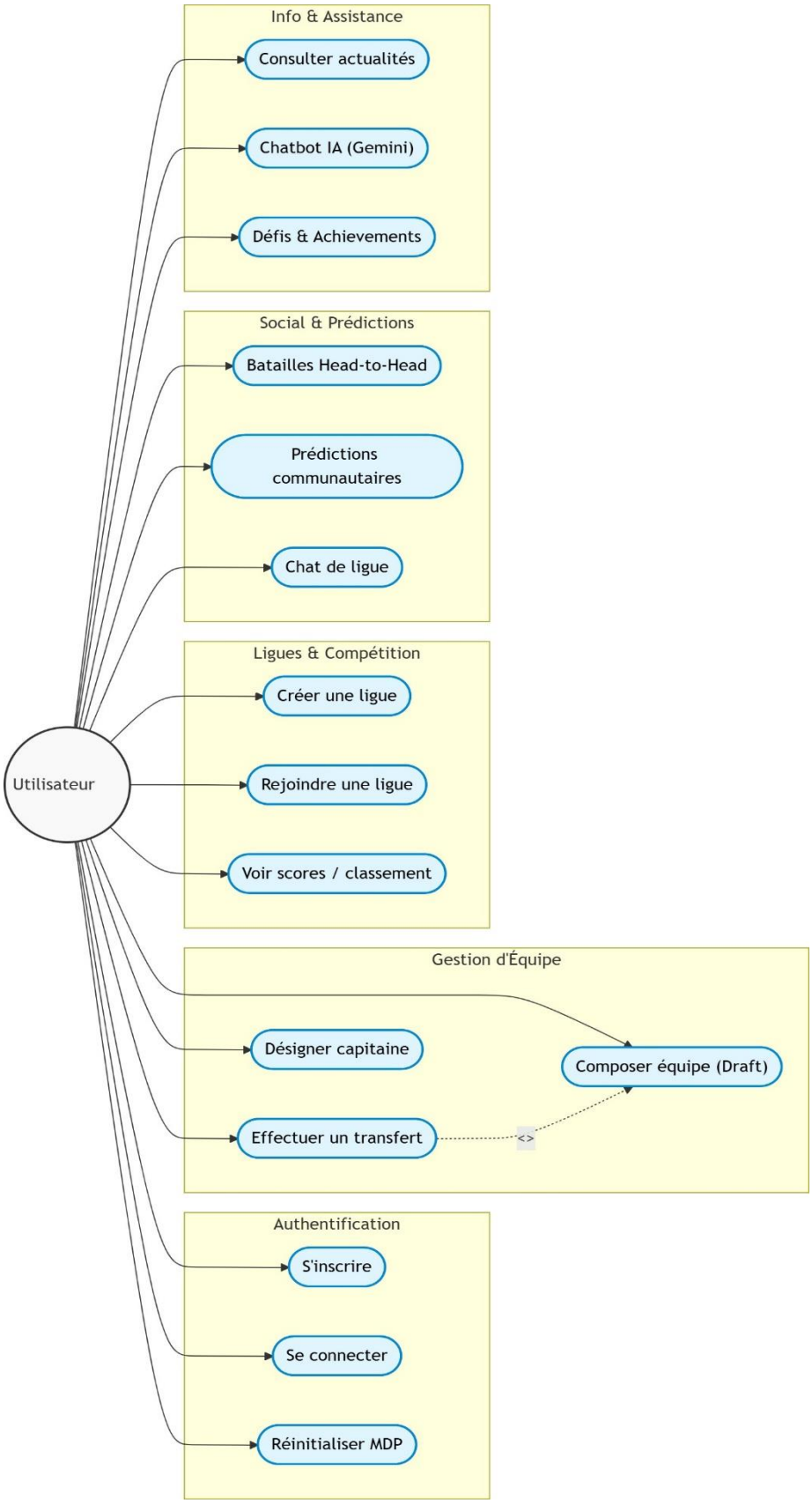


FIGURE 3.2 – Diagramme de cas d'utilisation — Utilisateur

Ce diagramme affiche toutes les actions réalisables par un utilisateur standard, telles que l'inscription, la gestion de l'équipe, la participation aux batailles et l'interaction avec le chatbot IA.

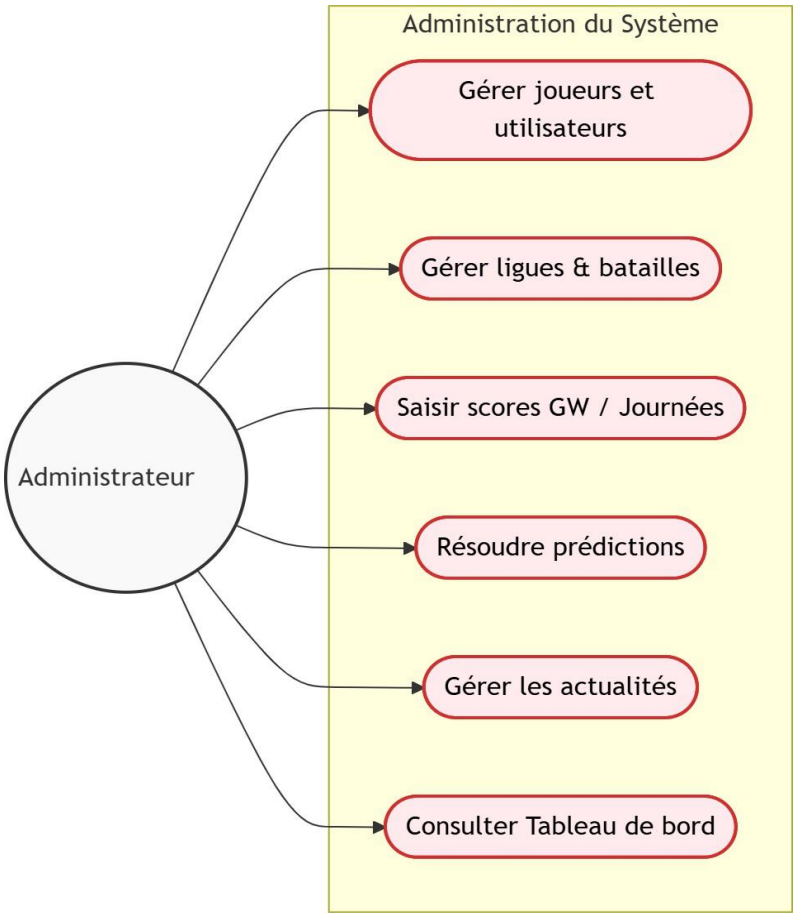


FIGURE 3.3 – Diagramme de cas d'utilisation — Administrateur

Ce diagramme présente les privilèges de l'administrateur, incluant la gestion du catalogue des joueurs, la validation des scores après chaque journée et la résolution des prédictions communautaires.

Le diagramme met en évidence cinq grands domaines fonctionnels :

- **Authentification** : inscription, connexion, réinitialisation du mot de passe.
- **Équipe** : composer son équipe, effectuer des transferts, désigner un capitaine.
- **Compétition** : créer/rejoindre des ligues, participer aux batailles, consulter les journées.
- **Profil & Joueurs** : consulter le profil d'un joueur, suivre ses statistiques.
- **Information & Assistance** : actualités, chatbot IA.
- **Administration** : gérer les utilisateurs, joueurs, scores, ligues et prédictions.

6. Modélisation structurelle — Schéma de base de données

La base de données PostgreSQL du projet comporte **20 tables** organisées autour de l'entité centrale users. Le tableau ci-dessous présente les tables principales avec leur rôle et les colonnes clés :

Table	Rôle & Colonnes principales
users	Comptes utilisateurs : id, username, email, password_hash, total_points, xp, rank
teams	Équipes fantasy : id, user_id, league_id, budget, formation, captain_id, players (JSONB)
players	Catalogue joueurs CdM : id, name, position, club, nationality, price, points, stats (JSONB)
leagues	Ligues : id, name, owner_id, max_members, is_private, invite_code, season, status
league_members	Appartenance ligue-utilisateur : league_id, user_id, points
battles	Batailles H2H : id, league_id, home_team_id, away_team_id, home_score, away_score, status
gameweeks	Journées de jeu : id, number, season, status, deadline
player_scores	Scores réels par journée : goals, assists, clean_sheet, saves, yellow_cards, total_points
predictions	Prédictions : predicted_top_scorer, predicted_winner_nation, predicted_goals, bonus_points
chat_messages	Messages du chat de ligue : league_id, user_id, message, created_at
refresh_tokens	Tokens de rafraîchissement JWT : user_id, token, expires_at
user_challenges	Défis complétés : user_id, challenge_key, challenge_type, xp_earned
user_achievements	Achievements débloqués : user_id, achievement_key, unlocked_at
password_reset_tokens	Tokens de reset MDP : user_id, token_hash, expires_at

TABLE 3.4 – Schéma de la base de données

Ce tableau décrit l'architecture relationnelle de la base de données PostgreSQL en listant les rôles et les colonnes clés des principales tables du schéma.

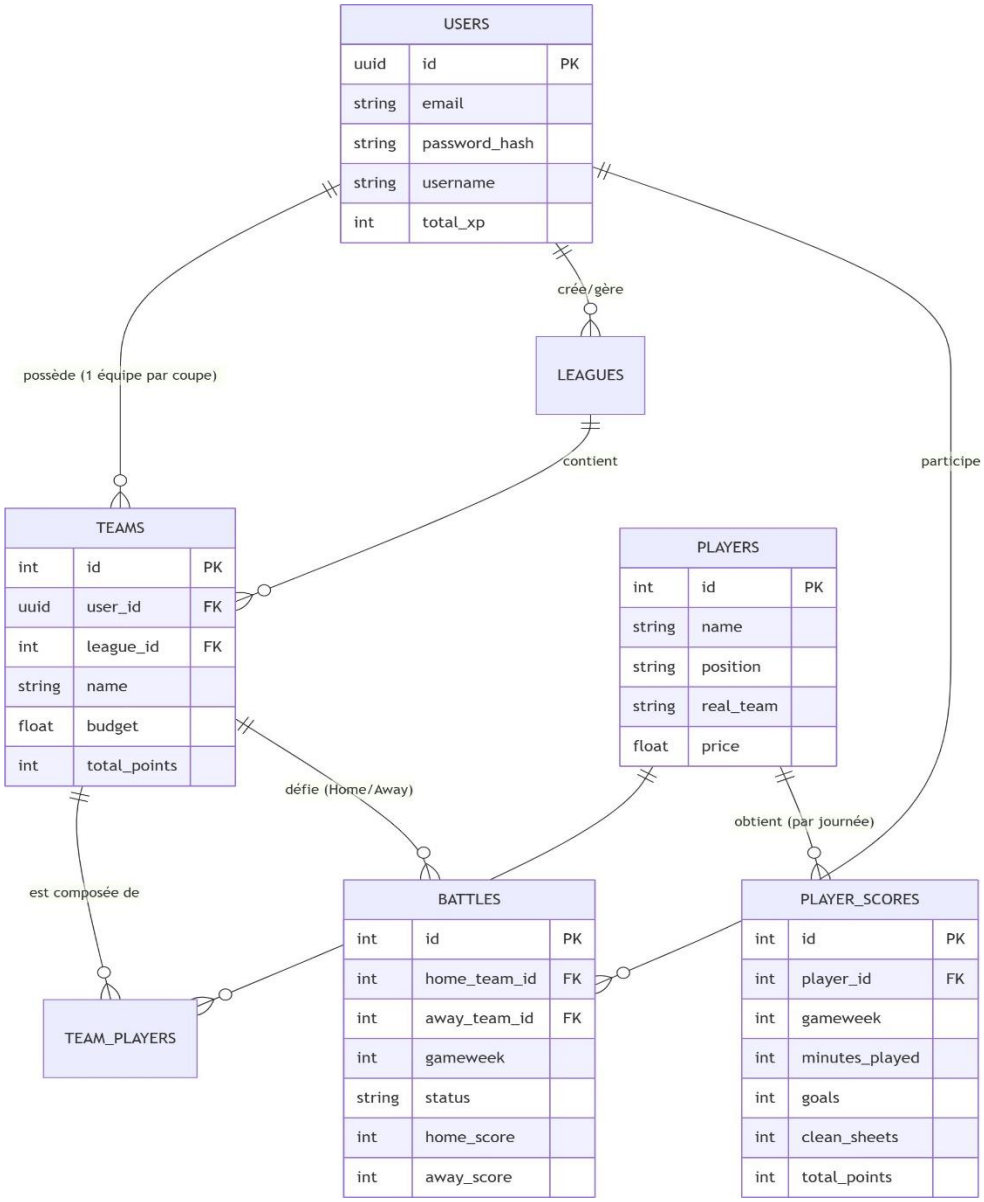


FIGURE 3.4 – Modèle Conceptuel de Données (MCD)

Ce schéma illustre la structure de la base de données relationnelle, mettant en évidence les liaisons entre les utilisateurs, leurs équipes virtuelles, les ligues et les statistiques des joueurs réels.

Le Modèle Conceptuel de Données (Figure 3.4) structure les relations clés suivantes :

- **Utilisateurs et Équipes** : Un utilisateur (USERS) peut posséder plusieurs équipes (TEAMS), à raison d'une seule équipe par ligue.
- **Composition d'équipe** : La table associative TEAM_PLAYERS lie les équipes virtuelles aux joueurs réels du catalogue (PLAYERS), représentant l'effectif sélectionné de 15 joueurs.
- **Performance des joueurs** : Les statistiques réelles de chaque joueur (buts, cartons,

minutes jouées) et ses points calculés pour chaque journée de la compétition sont historisés dans la table PLAYER_SCORES.

— **Batailles Head-to-Head** : Les affrontements directs entre managers se font via la table BATTLES, qui associe deux équipes (hôte et invitée) pour une journée (gameweek) précise.

7. Modélisation structurelle — Diagramme de classes

Le diagramme de classes présente la structure logique du système en décrivant les classes, leurs attributs, leurs méthodes ainsi que les associations existant entre elles. Il est structuré en quatre grands packages fonctionnels :

— **Comptes utilisateurs (User Accounts)** : gère les données d'authentification et de profil des utilisateurs.

— **Ligues, équipes & joueurs (Leagues, Teams & Players)** : représente le cœur du jeu de fantasy (gestion des ligues privées/publiques, composition des équipes virtuelles de 15 joueurs et catalogue des joueurs réels).

— **Compétition (Competition)** : gère le calendrier des journées de jeu (*Gameweeks*), les scores réels des joueurs, les duels Head-to-Head (*Battles*) et les prédictions communautaires.

— **Communication** : gère la messagerie instantanée Socket.io au sein de chaque ligue.

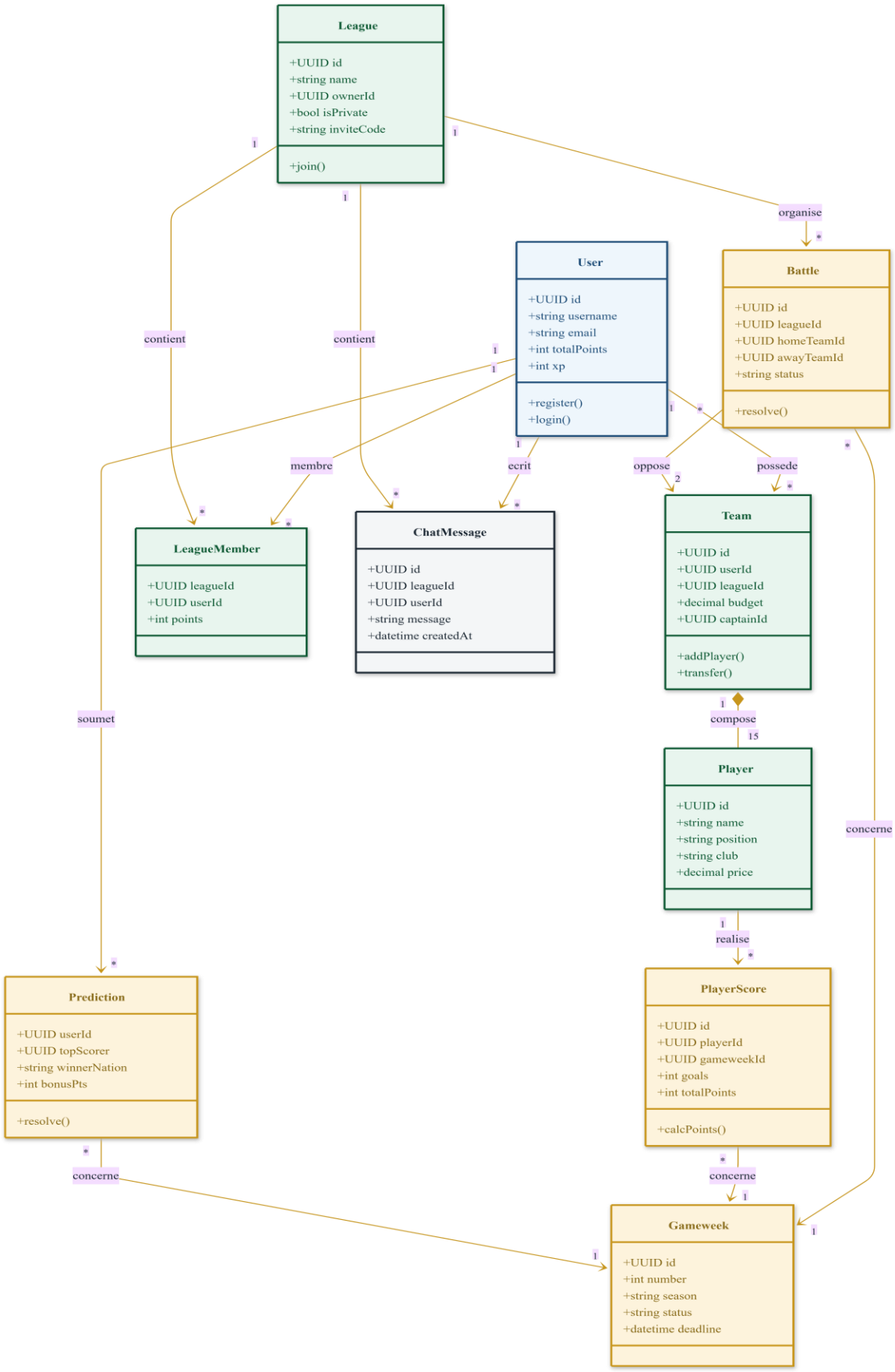


FIGURE 3.6 – Diagramme de classes du système WC Fantasy

Ce diagramme de classes illustre l'organisation orientée objet du backend de WC Fantasy, montrant comment les entités collaborent pour implémenter les règles du jeu (transferts, calcul de points, duels, prédictions et messagerie).

Les principales classes et leurs relations sont décrites ci-dessous :

- **User** : Représente le compte d'un utilisateur. Il possède des méthodes de gestion de compte (`register()`, `login()`) et est associé à ses équipes (`Team`) ainsi qu'à ses prédictions.
- **Team** : Représente l'équipe virtuelle d'un utilisateur pour une ligue spécifique. Elle a un budget limité (100M) et contient exactement 15 joueurs (`Player`). Ses méthodes clés sont `addPlayer()` et `transfer()`.
- **League** : Représente un groupe de compétition (publique ou privée) créé par un utilisateur. Une ligue organise des batailles hebdomadaires (`Battle`) entre les équipes de ses membres.
- **Player** : Représente un joueur de football réel participant à la Coupe du Monde 2026.
- **PlayerScore** : Contient les statistiques et les points d'un joueur réel pour une journée (`Gameweek`) donnée. La méthode `calcPoints()` calcule le score selon un barème précis.
- **Battle** : Représente un match direct (Head-to-Head) opposant deux équipes de la même ligue pendant une journée. La méthode `resolve()` compare les scores pour désigner le vainqueur.
- **Prediction** : Permet aux utilisateurs de parier sur les résultats ou le meilleur buteur d'une journée afin de remporter des points bonus XP.
- **ChatMessage** : Gère l'historique des conversations partagées dans le chat d'une ligue.

8. Modélisation dynamique — Diagrammes d'activité

Quatre diagrammes d'activité illustrent les scénarios les plus importants du système :

8.1. Diagramme 1 : Authentification utilisateur

Ce diagramme décrit le flux d'authentification, depuis l'ouverture de l'application jusqu'au chargement de l'écran principal. Il couvre les cas d'inscription (POST /api/auth/register) et de connexion (POST /api/auth/login), avec la vérification du token JWT en cache et la gestion des erreurs de validation.

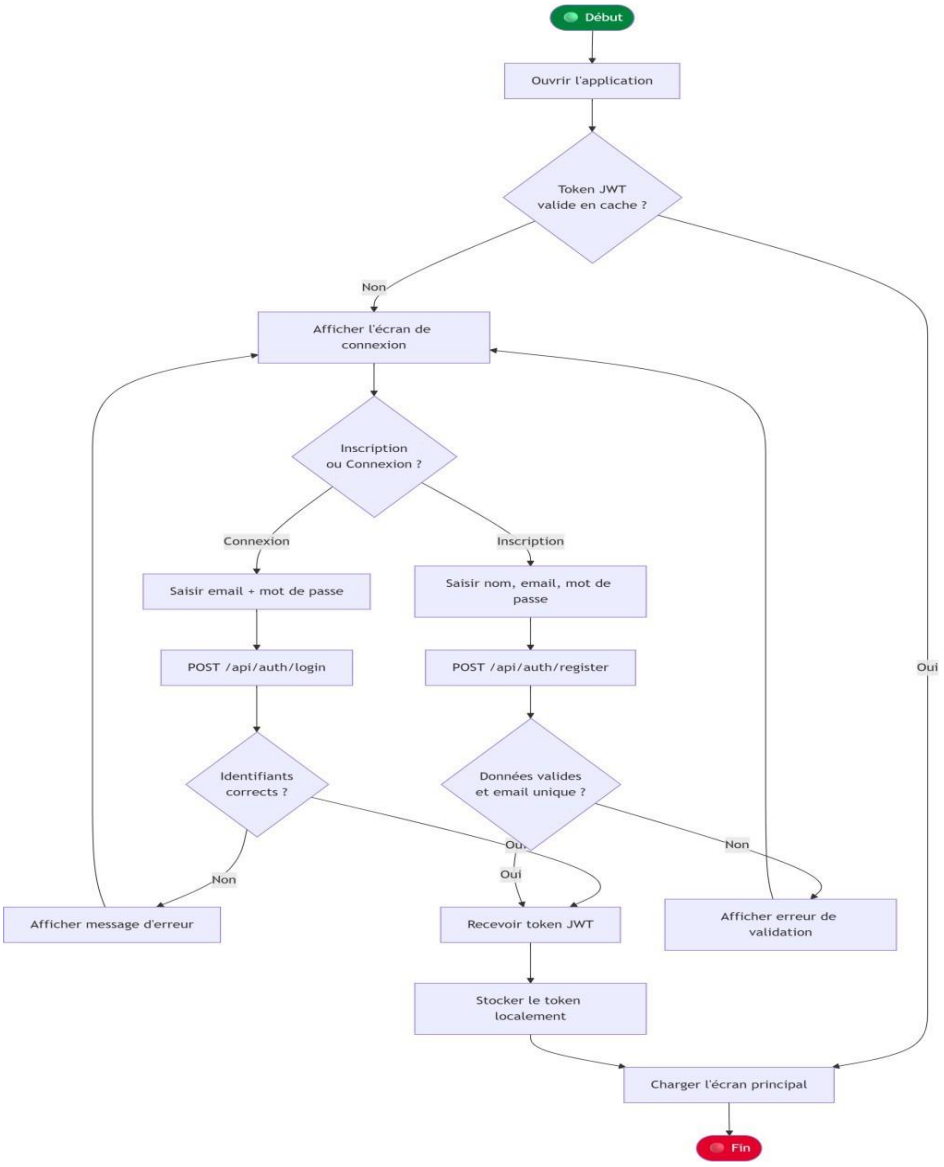


FIGURE 3.6 – Diagramme d'activité — Authentification utilisateur

Ce logigramme détaille les étapes séquentiellement suivies par le système lors de la tentative de connexion ou d'inscription d'un utilisateur, incluant la vérification et la génération des tokens JWT.

8.2. Diagramme 2 : Composition de l'équipe (Draft)

Ce diagramme détaille le processus de composition d'équipe. L'utilisateur consulte le catalogue filtrable des joueurs, sélectionne un joueur, et décide de l'ajouter ou le retirer selon la disponibilité du budget et les contraintes de positions. Chaque opération est vérifiée côté serveur avec un verrou transactionnel FOR UPDATE pour éviter les race conditions.

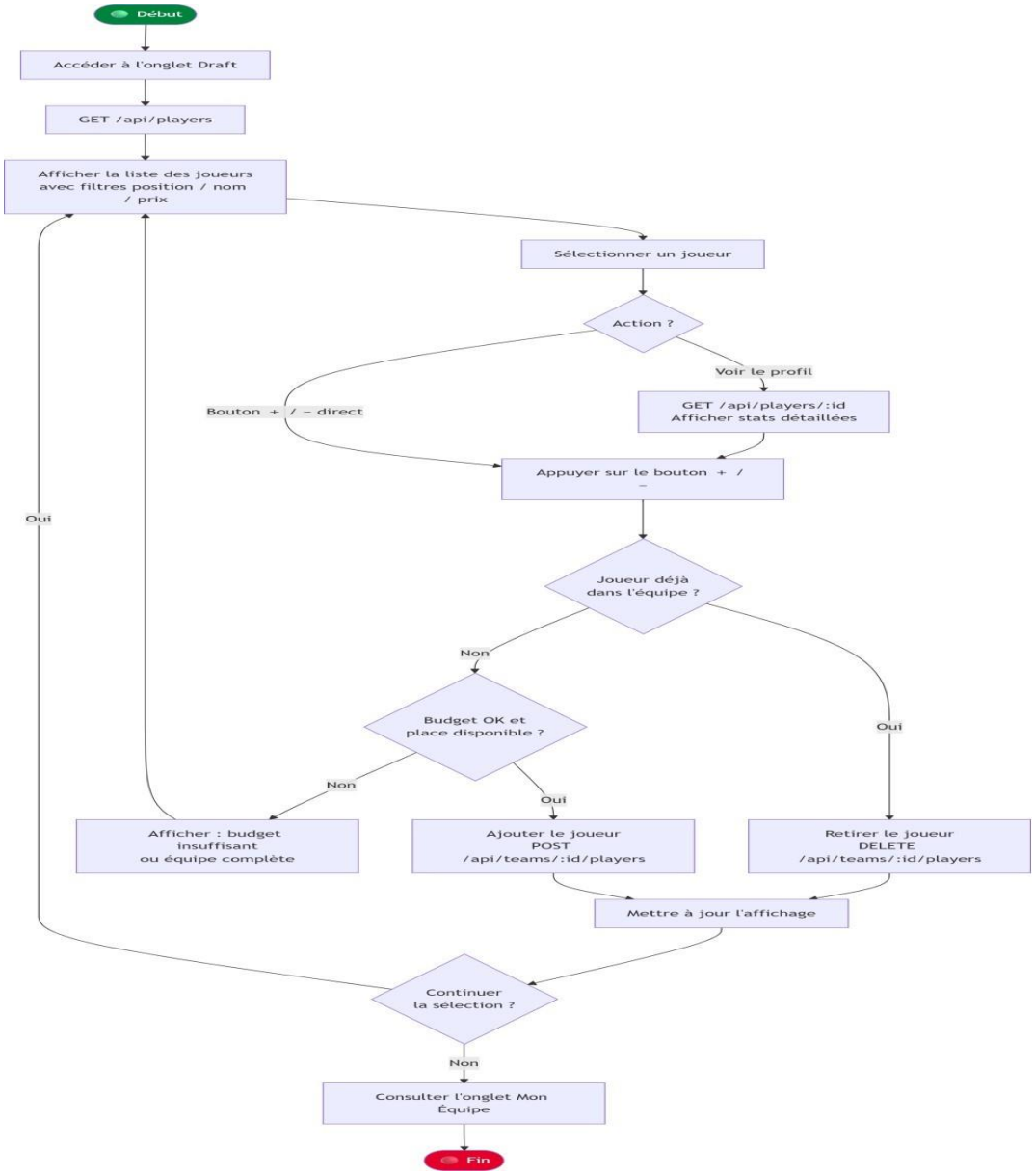


FIGURE 3.7 – Diagramme d'activité — Composition de l'équipe (Draft)

Ce processus décrit la logique métier exécutée lorsqu'un utilisateur modifie son équipe. Il montre comment le système vérifie le budget disponible et applique les verrous transactionnels pour éviter les erreurs concurrentes.

8.3. Diagramme 3 : Prédictions communautaires

Avant chaque journée de jeu, les utilisateurs peuvent soumettre trois prédictions : le meilleur buteur, la nation la plus performante et le nombre total de buts. Ce diagramme montre le flux de soumission, la vérification du statut de la journée et la confirmation. Une fois la journée terminée, l'admin résout les prédictions et distribue les bonus XP.

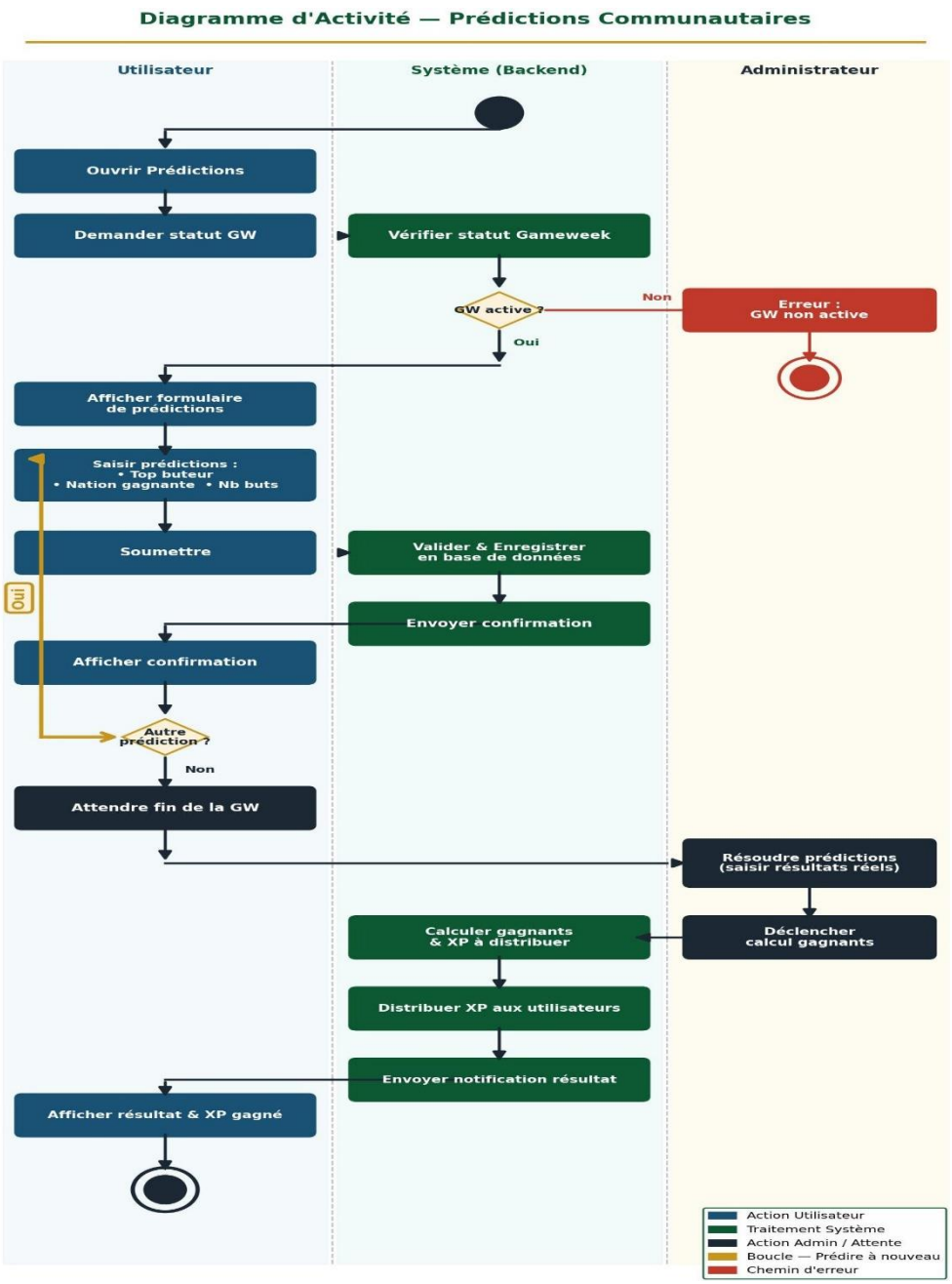


FIGURE 3.8 – Diagramme d'activité — Prédictions communautaires

Ce diagramme illustre le parcours de l'utilisateur pour soumettre une prédiction sur le vainqueur ou le meilleur buteur, ainsi que le processus de validation par l'administrateur en fin de match.

8.4. Diagramme 4 : Résolution des batailles

En fin de journée, les scores des équipes home et away sont calculés (buts×4, passes×3, clean sheets×4, etc.), comparés, et le résultat est enregistré dans la table battles. Une notification push est envoyée aux deux équipes, et l'historique est consultable immédiatement via GET /api/battles.

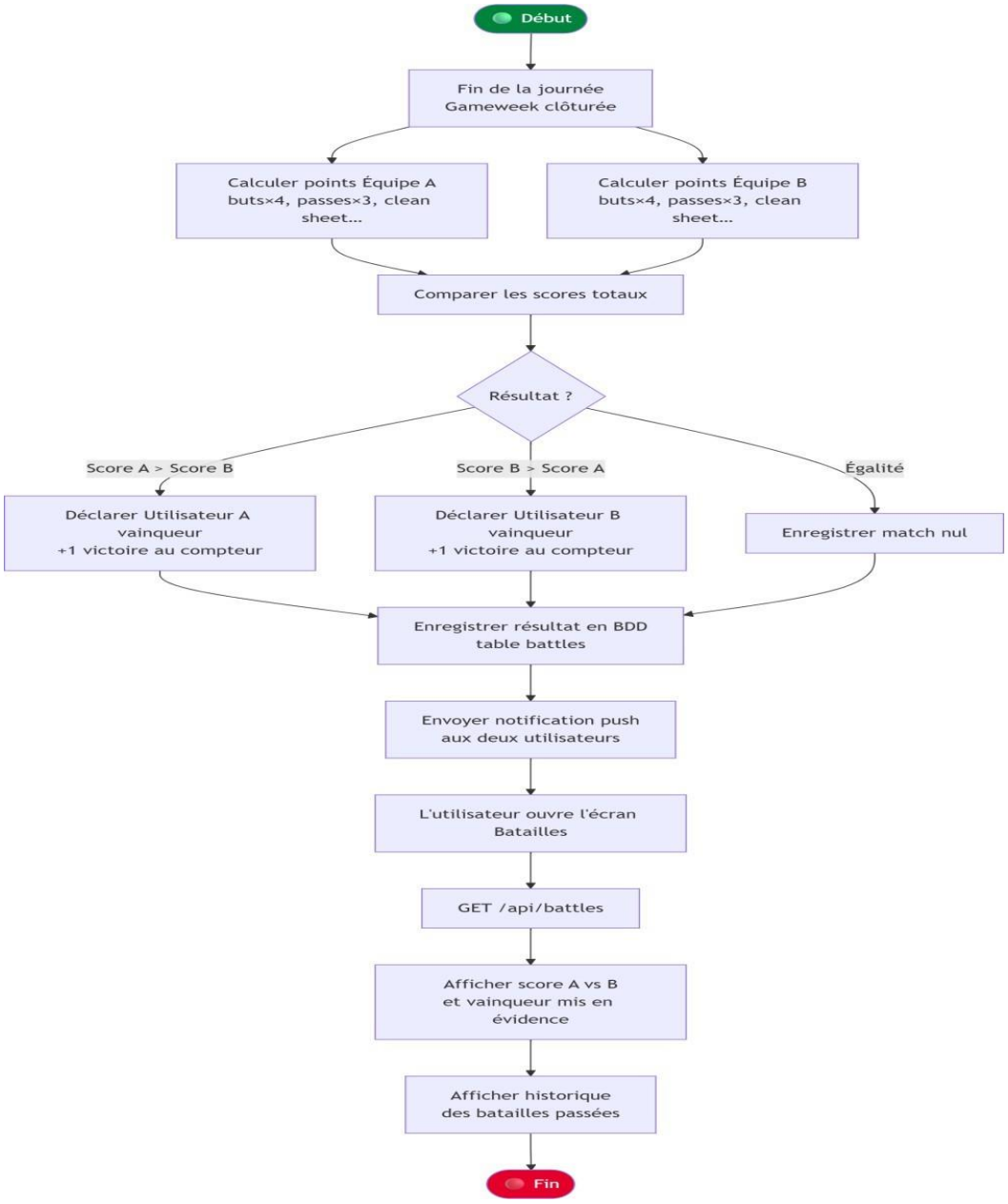


FIGURE 3.9 – Diagramme d'activité — Résolution des batailles

Cette figure résume l'algorithme de calcul qui compare les points de deux équipes s'affrontant dans une bataille Head-to-Head, avant d'enregistrer le résultat final et de notifier les vainqueurs.

8.5. Diagramme de Séquence : Synchronisation des Scores

Ce diagramme illustre le flux technique de mise à jour automatique des scores et points via la tâche Cron et les WebSockets :

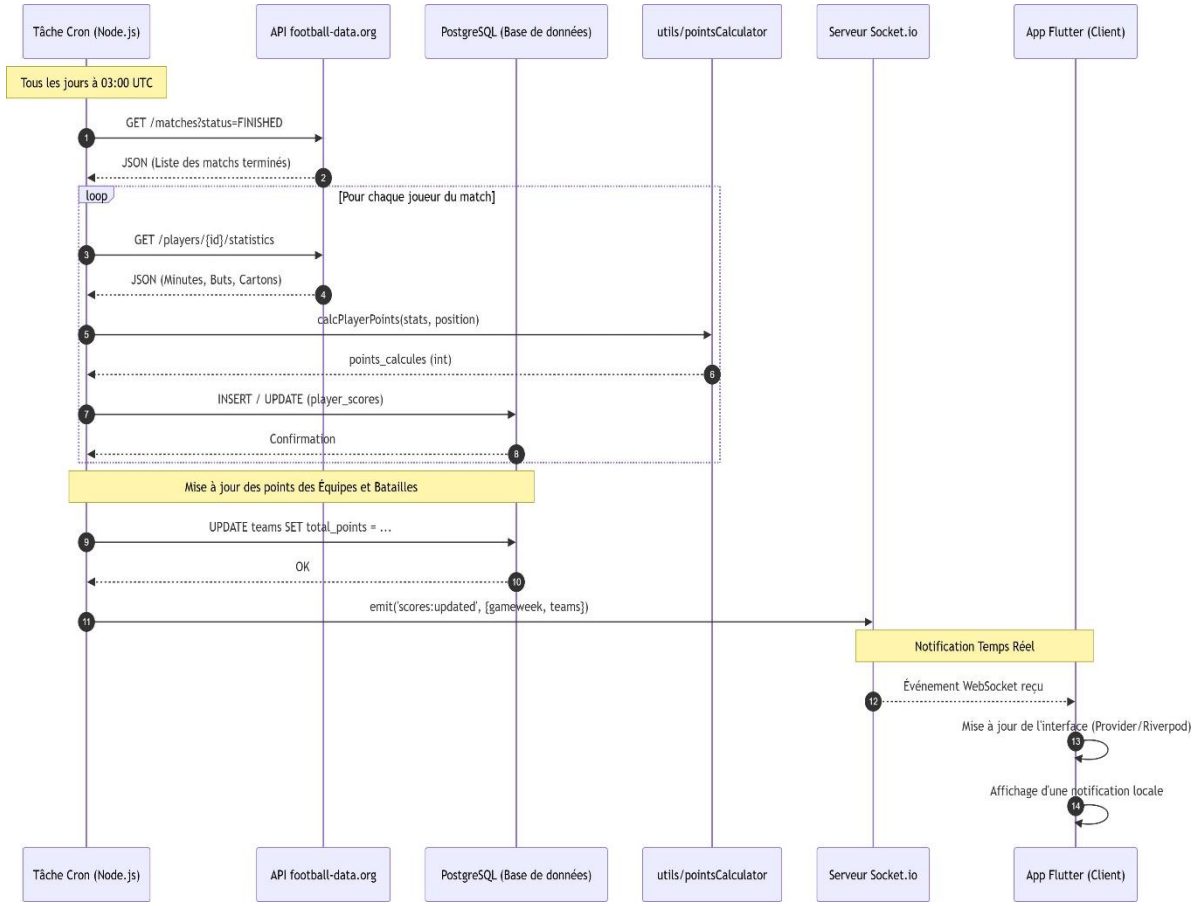


FIGURE 3.10 – Diagramme de séquence — Processus de synchronisation des scores

Ce diagramme de séquence décrit les échanges d'informations chronologiques entre le serveur, l'API externe (football-data) et l'application cliente lors de la mise à jour automatique et quotidienne des scores.

Le processus de synchronisation (Figure 3.10) se déroule comme suit :

1. **Déclenchement temporel** : Une tâche Cron programmée sur le serveur Node.js s'exécute quotidiennement à 03 :00 UTC.
2. **Récupération des résultats réels** : Le serveur interroge l'API externe football-data.org pour obtenir la liste des matchs finalisés (FINISHED).
3. **Extraction et Calcul** : Pour chaque match, les statistiques individuelles de performance des joueurs sont extraites et envoyées au module pointsCalculator qui applique le barème métier selon la position du joueur.
4. **Persistance** : Les points calculés sont mis à jour ou insérés dans la table player_scores de la base de données PostgreSQL.
5. **Diffusion temps réel** : Les points cumulés des équipes virtuelles et les résultats des batailles sont recalculés. Le serveur émet immédiatement un événement de notification scores:updated via le serveur Socket.io pour actualiser en temps réel l'interface client de tous les utilisateurs connectés.

9. Choix technologiques

Le tableau ci-dessous récapitule chaque technologie retenue et sa justification :

Technologie	Justification du choix
Node.js + Express.js [16]	Ecosystème JS unifié, asynchrone natif (I/O non bloquant), nombreux packages (jwt, bcrypt, pg, socket.io), performance élevée pour des API REST.
PostgreSQL [2]	SGBDR ACID complet, support JSON/JSONB pour stocker les compositions d'équipes, UUID natifs, contraintes CHECK pour l'intégrité (budget ≥ 0).
Redis [3]	Cache en mémoire ultra-rapide (< 1ms), réduction drastique des requêtes DB répétitives, TTL automatique, pub/sub possible.
Socket.io [4]	Abstraction WebSocket avec fallback HTTP long-polling, rooms nommées, authentification middleware JWT, reconnexion automatique.
Flutter + Dart [5]	Un seul codebase pour iOS et Android, performances natives (moteur Skia), Riverpod pour la gestion d'état réactive, widgets riches.
Riverpod [6]	Gestion d'état compilée, thread-safe, testable sans BuildContext, supporte l'async avec AsyncNotifier.
JWT [7]	Stateless, scalable horizontalement, rotation sécurisée du refresh token, révocation possible.
Brevo SMTP [10]	Service email gratuit (300 emails/jour), fiable, supporte les templates HTML, simple à intégrer via Nodemailer [15].
Google Gemini API [9]	LLM performant, API gratuite (niveau standard), réponses contextuelles pour le chatbot football.
football-data.org [8]	API officielle FIFA/UEFA, couvre la Coupe du Monde, données structurées (matchs, joueurs, scores), plan gratuit disponible.

TABLE 3.5 – Justification des choix technologiques

Ce tableau expose la justification technique et fonctionnelle associée au choix de chaque brique technologique retenue pour le projet.

Conclusion

Ce chapitre a formalisé l'ensemble des besoins fonctionnels et non fonctionnels de WC Fantasy, identifié les acteurs, présenté l'architecture globale et les modèles UML qui guident le développement. Les choix technologiques retenus — Node.js, PostgreSQL, Redis, Socket.io, Flutter — constituent une stack moderne, cohérente et adaptée aux contraintes du projet. Le chapitre suivant présente la réalisation concrète de ce système et les résultats obtenus.

Chapitre 4

Développement, Validation et Résultats

Introduction

Ce chapitre présente la réalisation concrète du système WC Fantasy. Il détaille l'environnement de développement, les fonctionnalités implémentées avec leurs extraits de code significatifs, les tests réalisés pour valider le fonctionnement, les difficultés rencontrées et les solutions apportées.

1. Environnement de développement

Le développement a été réalisé sur Windows 11, et les outils suivants ont été mobilisés pour la conception et la réalisation du projet :

Outil / Logiciel	Version & Rôle
Visual Studio Code	IDE principal — extensions ESLint, Prettier, Flutter, GitLens
Android Studio	Émulateur Android et compilation de l'application Flutter mobile
Node.js	v20 LTS — runtime JavaScript côté serveur
npm[17]	v10 — gestionnaire de paquets Node.js
Flutter SDK	v3.x — framework mobile cross-platform
Dart SDK	v3.x — langage de programmation (inclus avec Flutter)
PostgreSQL	v16 — système de gestion de base de données relationnelle
pgAdmin 4	Interface graphique d'administration PostgreSQL
Redis	v7 — serveur de cache en mémoire
Git	Contrôle de version — dépôt local et hébergement distant (GitHub)
Postman	Tests et documentation des endpoints REST de l'API
TablePlus	Visualisation et requêtes directes sur la base de données

TABLE 4.1 – Environnement de développement et pile technologique

Ce tableau récapitule les logiciels, outils et frameworks utilisés avec leurs rôles respectifs dans le cycle de développement du projet, couvrant l'IDE, le serveur, les bases de données et les outils de test.

2. Implémentation du système

2.1. Structure du projet backend

Le backend est organisé selon le pattern architectural **Modèle-Vue-Contrôleur (MVC)**, adapté pour les API REST (où la "Vue" est remplacée par des réponses JSON). Ce choix garantit une séparation claire des responsabilités entre le routage, la logique métier et l'accès aux données.

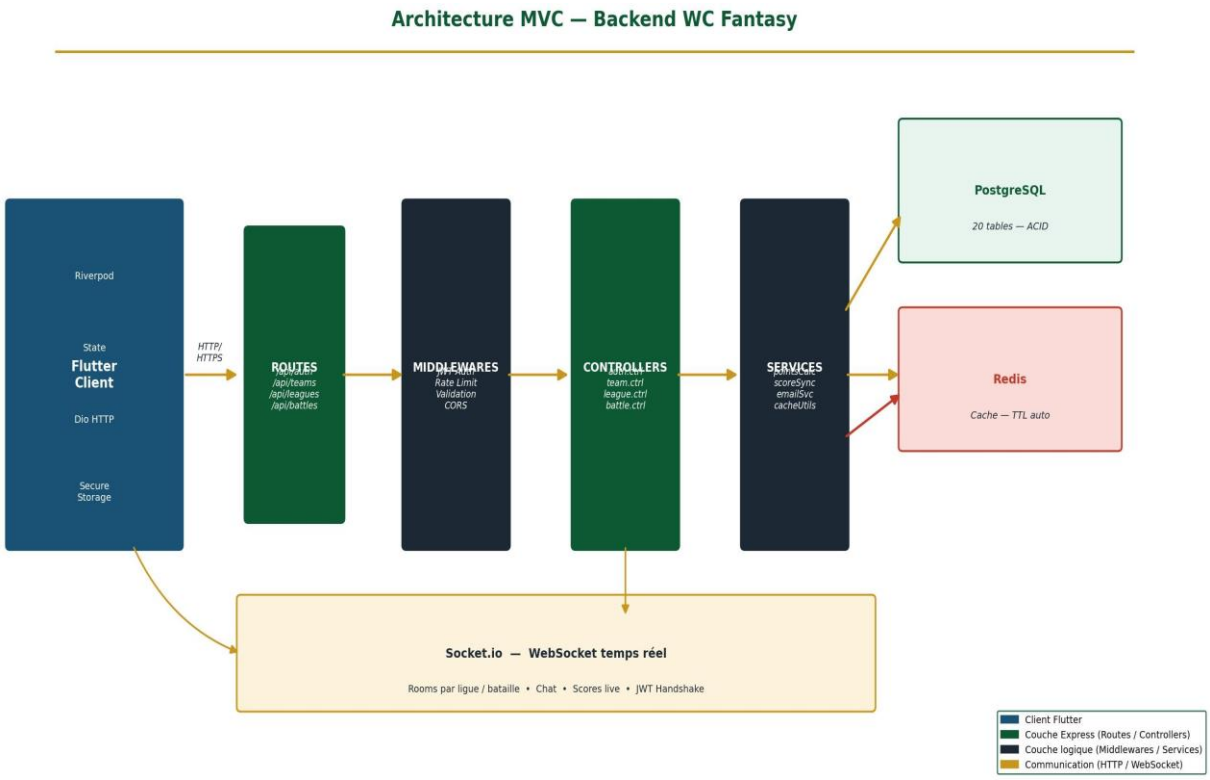


FIGURE 4.1 – Architecture MVC du backend WC Fantasy

Ce schéma montre la répartition des dossiers du code source selon le motif MVC, où les "Routes" dirigent vers les "Controllers", qui s'appuient eux-mêmes sur les "Services" et "Models" pour manipuler les données de manière modulaire.

Cette arborescence se traduit concrètement par l'organisation des dossiers suivante :

Dossier	Contenu
controllers/	Endpoints REST (logique de requête/réponse)
services/	Logique métier (calcul points, sync scores, etc.)
models/	Schémas ORM + migrations SQL
middleware/	Auth (JWT), rate-limiting, validation
socket/	Événements WebSocket (Socket.io)
routes/	Définition des routes API
utils/	Helpers (cache, email, logging, pointsCalculator)
migrations/	Scripts SQL de création/modification des tables

TABLE 4.2 – Structure du projet

Ce tableau présente l'arborescence structurelle du backend en détaillant le rôle spécifique de chaque répertoire dans l'architecture MVC adoptée.

2.2. Authentification et sécurité

Le système d'authentification repose sur une double stratégie de tokens JWT. Lors de la connexion, le serveur émet un **access token** (validité 7 jours) et un **refresh token** (validité 30 jours). L'access token est vérifié à chaque requête protégée par le middleware auth.middleware.js. Lorsque l'access token expire, l'application Flutter utilise automatiquement le refresh token pour en obtenir un nouveau, de façon transparente pour l'utilisateur.

Les mots de passe sont hachés avec **bcrypt** (10 rounds de salt) avant stockage. La réinitialisation du mot de passe génère un token aléatoire sécurisé (SHA-256), stocké haché en base avec une expiration d'une heure, et transmis par email via Brevo SMTP. Le rate limiting global (1 000 req/15min par IP) et un rate limiter spécifique sur la route de connexion (10 tentatives/15min) protègent contre les attaques par force brute (recommandations de sécurité OWASP[11]).

Côté Flutter, les tokens sont stockés dans FlutterSecureStorage (chiffré par le keychain iOS/Keystore Android). L'intercepteur Dio gère automatiquement le rafraîchissement transparent et la déconnexion en cas d'échec du refresh.

2.3. Gestion de l'équipe et des transferts

La constitution d'une équipe est soumise à plusieurs contraintes métier strictement enforced côté serveur :

- Budget maximum de 100M, avec contrainte CHECK (budget ≥ 0) en base de données pour éviter les dépassements.
- Maximum 15 joueurs par équipe (1 GK, 4 DEF, 4 MID, 3 FWD + 3 remplaçants selon la formation).
- Maximum 3 joueurs par club réel pour respecter l'esprit du jeu.
- Les opérations d'ajout et de suppression utilisent des transactions PostgreSQL avec SELECT FOR UPDATE pour éliminer toute race condition en cas d'appels concurrents.

Le système de transferts valide que le joueur sortant appartient bien à l'équipe, que le joueur entrant n'y est pas déjà, et que le solde budgétaire reste positif après l'opération.

Un cache Redis invalidé à chaque modification assure que l'état retourné est toujours à jour.

2.4. Calcul des points fantasy

Le barème de points est centralisé dans le module utils/pointsCalculator.js, source unique de vérité utilisée à la fois par le controller de saisie des scores (gameweek.controller.js) et par la tâche de synchronisation automatique (scoreSync.job.js). Ce choix élimine toute divergence possible entre les deux modes de saisie.

Action / Événement	Gardien	Défenseur	Milieu	Attaquant
Jouer < 60 minutes	1 pt	1 pt	1 pt	1 pt
Jouer ≥ 60 minutes	2 pts	2 pts	2 pts	2 pts
But inscrit	10 pts	6 pts	5 pts	4 pts
Passe décisive	3 pts	3 pts	3 pts	3 pts
Clean sheet (≥60 min)	4 pts	4 pts	1 pt	—
Arrêt (par 3 arrêts)	1 pt	—	—	—
Penalty arrêté	5 pts	—	—	—
Penalty manqué	-2 pts	-2 pts	-2 pts	-2 pts
Carton jaune	-1 pt	-1 pt	-1 pt	-1 pt
Carton rouge	-3 pts	-3 pts	-3 pts	-3 pts
But contre son camp	-2 pts	-2 pts	-2 pts	-2 pts
Bonus capitaine	×2 tous points			

TABLE 4.3 – Barème de points fantasy

Ce tableau détaille la grille de calcul des scores attribuant des points positifs ou négatifs aux joueurs en fonction de leurs actions réelles sur le terrain, répartis selon leur poste de jeu.

2.5. Temps réel avec Socket.io

Le serveur Socket.io est initialisé dans socket/socket.js et partage le même serveur HTTP qu'Express. L'authentification est vérifiée au niveau du middleware de connexion (token JWT dans handshake.auth). Les rooms sont organisées par ligue (league:{id}) et par bataille (battle:{id}), permettant de cibler précisément les messages.

Les événements implémentés sont : league:join (avec vérification DB de l'appartenance), league:leave, battle:watch (avec vérification DB), draft:pick (diffusion dans la room de ligue), et chat:message (avec rate limit 1 msg/sec via Map en mémoire et vérification d'appartenance avant diffusion). La déconnexion nettoie automatiquement l'entrée de rate-limit.

2.6. Synchronisation automatique des scores

Une tâche cron (node-cron[14]) s'exécute **quotidiennement à 3h du matin UTC** pour synchroniser les scores réels des joueurs depuis l'API football-data.org. Elle récupère les matchs terminés de la journée en cours, extrait les statistiques individuelles (buts, passes,

minutes, cartons) et les insère ou met à jour dans la table player_scores. Le calcul de points utilise la même fonction calcPlayerPoints que la saisie manuelle, garantissant la cohérence.

2.7. Chatbot IA — Google Gemini

Le chatbot est accessible via POST /api/chatbot/chat. Il reçoit un message utilisateur, construit un prompt enrichi de contexte (compétition, équipes, actualités) et interroge l'API Google Gemini (modèle gemini-pro). La réponse est retournée en temps réel via Server-Sent Events (SSE) pour une expérience de streaming fluide. L'historique de conversation est maintenu côté client (Flutter) pour conserver le contexte sur plusieurs échanges.

3. Interfaces utilisateur

L'application Flutter est organisée selon une navigation par onglets principaux (Bottom Navigation Bar). Les différentes fonctionnalités sont illustrées par les captures d'écran ci-dessous.

3.1. Interfaces Mobiles

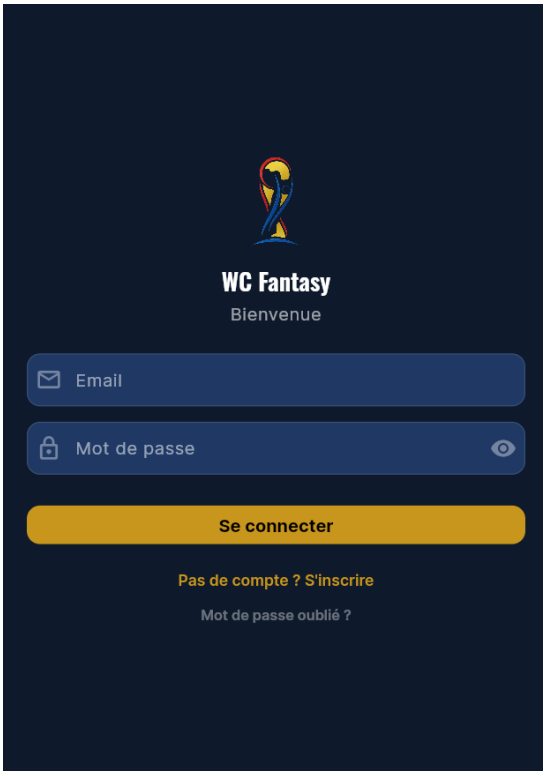


FIGURE 4.2 – Écran d'authentification

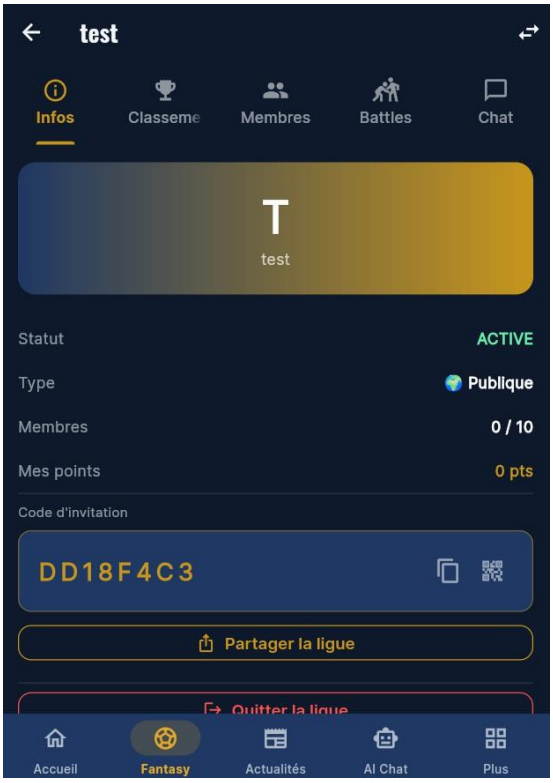


FIGURE 4.3 – Écran profil utilisateur

Ces captures d'écran montrent l'interface mobile Flutter. L'écran de gauche permet l'authentification sécurisée de l'utilisateur, tandis que celui de droite affiche le tableau de bord de son profil (incluant son rang, son niveau et son historique de matchs).

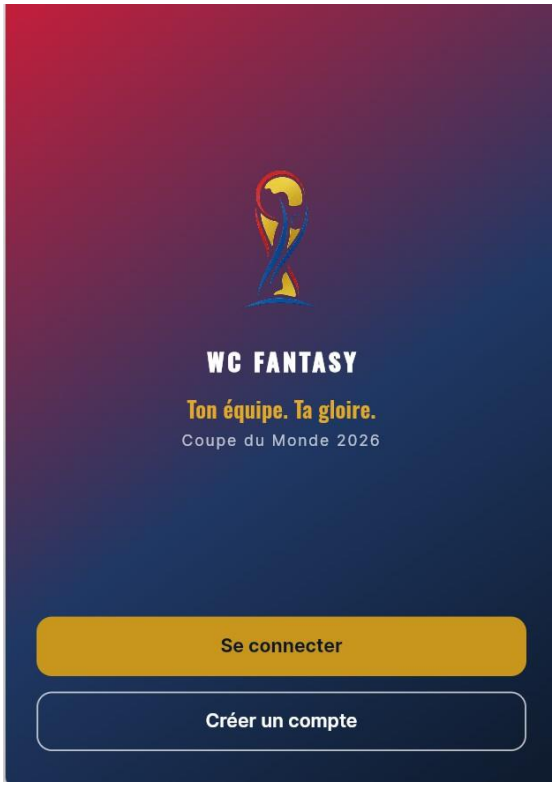


FIGURE 4.4 – Page d'accueil de l'application

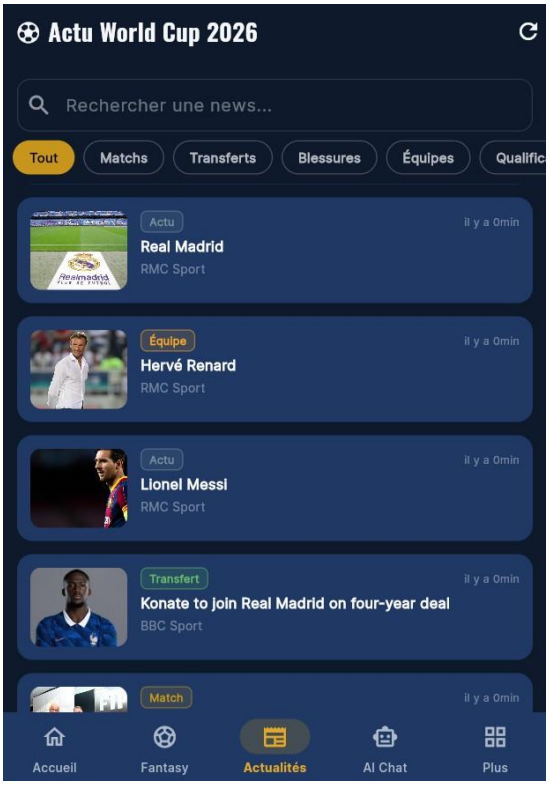


FIGURE 4.5 – Actualités et nouveaut

L'écran de gauche présente la page d'accueil avec les principales statistiques et raccourcis. L'écran de droite affiche le fil d'actualité permettant aux utilisateurs de suivre les dernières informations du tournoi.

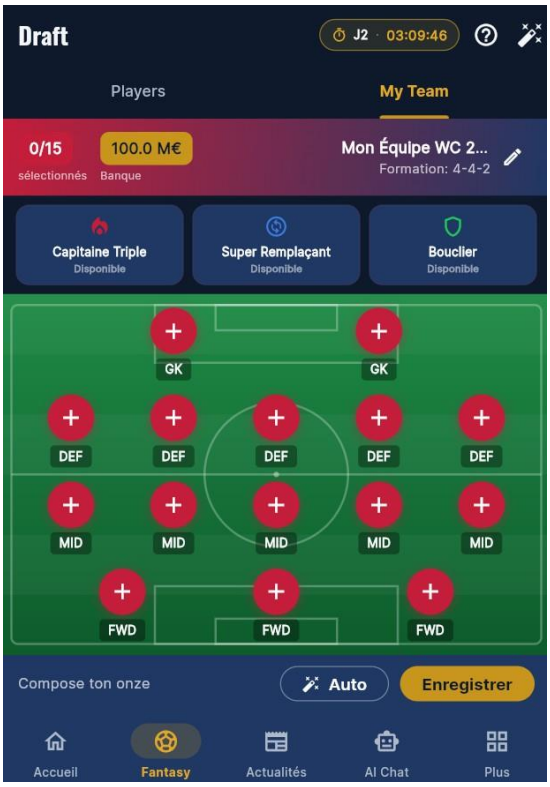


FIGURE 4.6 – Composition de l'équipe (Draft)

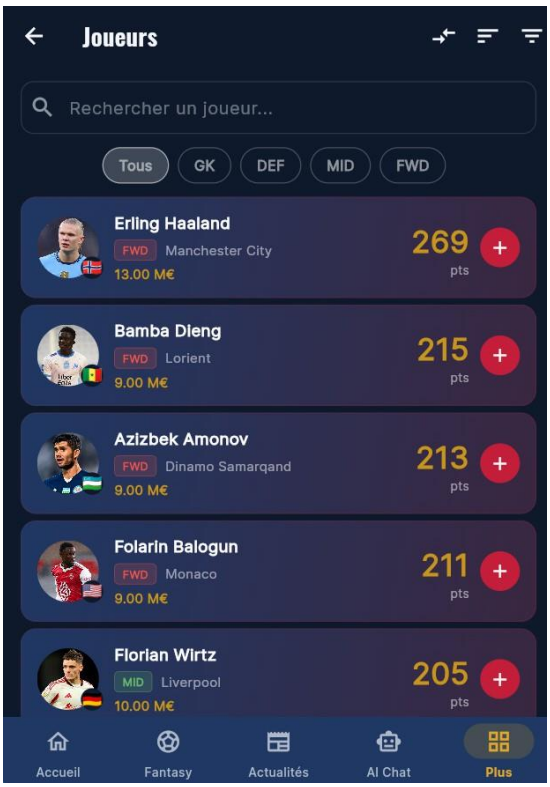


FIGURE 4.7 – Détails et statistiques d'un joueur

La capture de gauche présente l'interface du "Draft" où l'utilisateur compose son équipe sur un terrain interactif. Celle de droite montre la fiche détaillée d'un joueur, avec ses statistiques de performance et sa valeur.

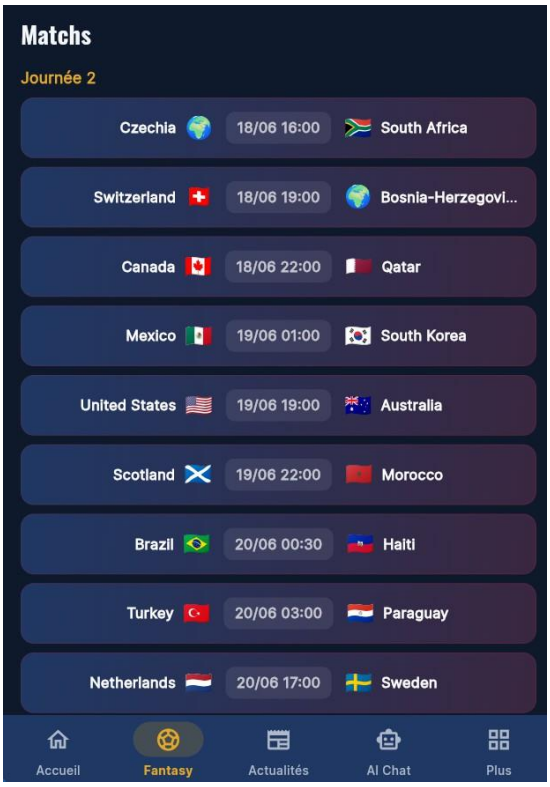


FIGURE 4.8 – Suivi des matchs

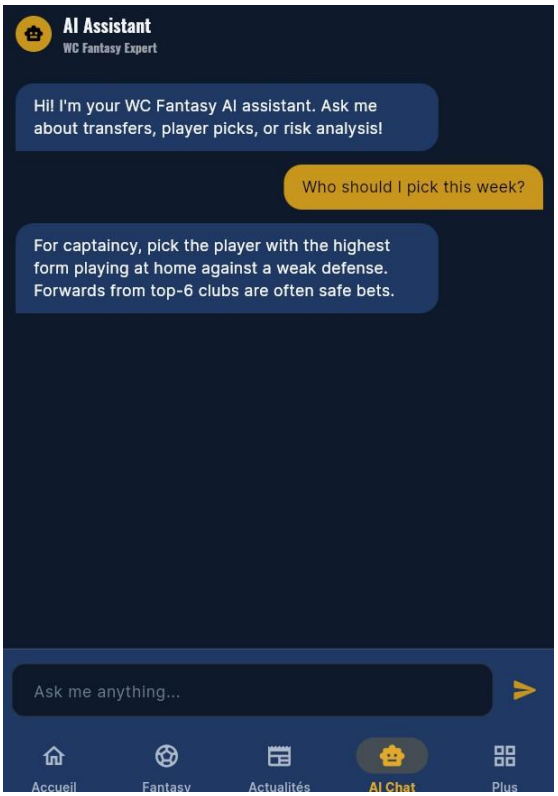


FIGURE 4.9 – Chatbot IA (Gemini)

L'interface de gauche est dédiée au suivi des matchs en direct, montrant les scores et les affrontements. À droite, l'interface du Chatbot IA intégré permet de formuler des recommandations ou de demander des statistiques sur le tournoi.

3.2. Interfaces d'Administration

Pour superviser la plateforme et saisir les résultats réels, l'administrateur dispose d'interfaces de gestion spécifiques. Les captures ci-dessous en illustrent deux exemples majeurs :

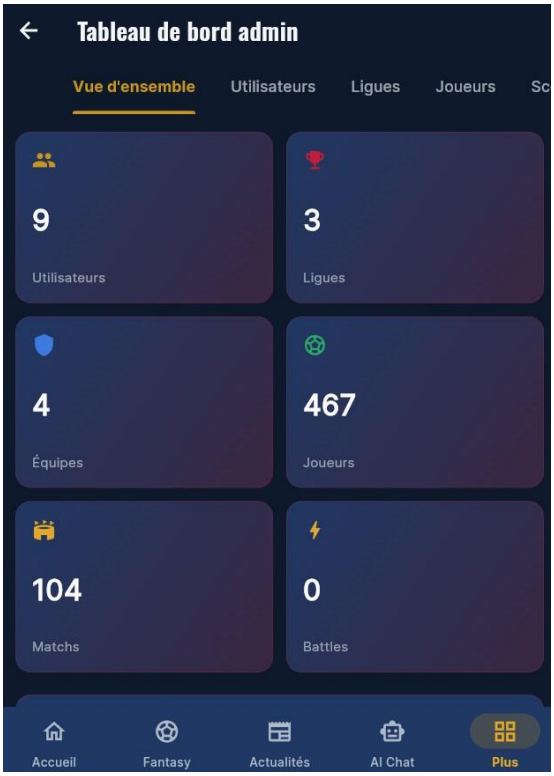


FIGURE 4.10 – Interface d'administra-tion —

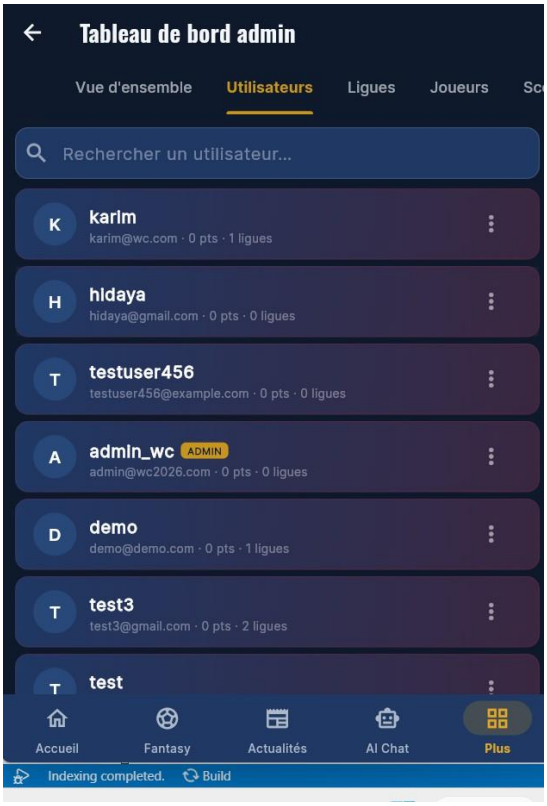


FIGURE 4.11 – Interface d'administration — Gestion des actualités

Saisie des scores Ces captures d'écran présentent l'interface d'administration de WC Fantasy. L'écran de gauche permet à l'administrateur d'enregistrer manuellement les scores réels des matchs et les performances des joueurs. L'écran de droite lui offre les outils nécessaires pour gérer, éditer et publier les actualités liées au tournoi.

4. Validation et tests

La validation du système a été réalisée via une combinaison de tests manuels avec Postman pour l'API et de tests fonctionnels sur l'application mobile. Le tableau ci-dessous présente les principaux scénarios testés :

Scénario testé	Résultat attendu	Résultat obtenu
Inscription avec email existant	Erreur 409 Conflict (email déjà utilisé)	Conforme
Connexion avec mauvais mot de passe	Erreur 401 Unauthorized + message clair	Conforme
Ajout d'un joueur hors budget	Erreur 400 : budget insuffisant	Conforme
Ajout concurrent de 2 joueurs (race)	Un seul ajout accepté, budget cohérent	Conforme
Rejoindre une ligue avec mauvais code	Erreur 404 : ligue introuvable	Conforme
Chat : flood 50 messages/seconde	Seul 1 message/seconde diffusé (rate limit)	Conforme
Token JWT expiré	Flutter rafraîchit automatiquement, requête repart	Conforme
Soumission prédiction GW terminée	Erreur 400 : GW déjà complétée	Conforme
Reset MDP token expiré (> 1h)	Erreur 400 : token invalide ou expiré	Conforme
Cache Redis : GET /api/news (2ème appel)	Réponse source : "cache", < 5 ms	Conforme
Saisie scores GK avec 6 arrêts	player_scores.saves = 6, +2 pts bonus	Conforme
Résolution prédiction : bonne nation	user.xp += 8, prediction.resolved = true	Conforme

TABLE 4.4 – Scénarios de tests réalisés

Ce tableau liste les différents cas limites et scénarios critiques testés au niveau de l'API et du client, en précisant les comportements attendus et validés pour chaque test.

5. Difficultés rencontrées et solutions apportées

Difficulté rencontrée	Solution apportée
Race condition sur le budget lors d'ajouts concurrents d'un joueur	Transaction PostgreSQL BEGIN/COMMIT + SELECT FOR UPDATE sur la ligne team
Les transferts échouaient : controller déstructurait playerOutId/playerInId alors que la route attendait playerOut/playerIn	Correction de la déstructuration : `const { playerOut: playerOutId, playerIn: playerInId } = req.body`
clearCache des news ne fonctionnait pas : redis n'est pas exporté depuis cache.js	Remplacement par `cacheDel()` (helper exporté), suppression de l'import redis inexistant
XP des défis/prédictions polluait le classement fantasy (total_points)	Ajout d'une colonne xp séparée (migration 010), réorientation des bonus vers xp uniquement
Double sérialisation JSON dans le cache des prédictions	Suppression du JSON.stringify manuel (cacheSet le fait déjà) et de JSON.parse (cacheGet parse déjà)
Flutter : logout effaçait l'URL d'API personnalisée (deleteAll)	Remplacement par suppression sélective : delete access_token + delete refresh_token
Ligues publiques sans code d'invitation (null) → impossibles à rejoindre	Génération systématique d'un code pour toutes les ligues (publiques et privées)
_getWeekStart() : mutation de la date via setDate() → bug aux fins de mois	Reconstruction sans mutation : `new Date(year, month, date + diff)`
psql non reconnu dans PowerShell pour les migrations	Ajout du bin PostgreSQL 16 au PATH
Column saves absente du INSERT player_scores malgré son utilisation dans calcPoints	Ajout de saves à la liste des colonnes INSERT et au SET de ON CONFLICT DO UPDATE

TABLE 4.5 – Difficultés rencontrées et solutions apportées

Ce tableau résume les principaux obstacles techniques rencontrés lors du développement de l'application et les solutions concrètes mises en œuvre pour les surmonter.

6. Limites du système

Certaines fonctionnalités n'ont pas été intégrées dans la version actuelle, principalement par contrainte de temps et de ressources :

— **Microservice IA (python/FastAPI)** : prévu pour des recommandations ML de joueurs basées sur les performances historiques. La route `/api/ai` est implémentée mais retourne des données simulées en attendant le déploiement du microservice.

— **Notifications push natives (FCM/APNs)** : l'architecture est prévue (champs device_token, service push), mais l'intégration Firebase Cloud Messaging n'est pas finalisée.
— **Mode hors-ligne complet** : un cache local limité est implémenté côté Flutter, mais la synchronisation complète hors-ligne n'est pas encore disponible.
— **Tests automatisés** : les tests unitaires (Jest côté backend, flutter_test côté mobile) sont partiellement écrits. Une couverture complète est prévue dans les versions futures.
— **Déploiement en production** : l'application fonctionne en environnement de développement local. La containerisation Docker est partiellement configurée (docker-compose.yml) mais pas encore déployée sur un serveur cloud.

Conclusion

Ce chapitre a présenté la réalisation complète de WC Fantasy : de l'organisation du code backend à l'implémentation des fonctionnalités clés (auth JWT, draft d'équipe, batailles temps réel, prédictions, chatbot IA), en passant par les tests de validation et les difficultés surmontées. Les résultats obtenus démontrent un **système fonctionnel, sécurisé et performant**, répondant à l'ensemble des objectifs fixés dans le chapitre de conception

Conclusion générale

Ce projet de fin d'année nous a permis de concevoir et développer WC Fantasy, une application mobile de fantasy football dédiée à la Coupe du Monde 2026. Partant d'un vide identifié dans l'offre existante — l'absence d'une plateforme mobile combinant fantasy sport, fonctionnalités sociales en temps réel et gamification pour cet événement mondial — nous avons proposé une solution complète et opérationnelle.

Sur le plan technique, le projet a mobilisé un large spectre de compétences : architecture backend avec Node.js/Express/PostgreSQL, développement mobile cross-platform avec Flutter/Dart, communications temps réel avec Socket.io, gestion du cache avec Redis, intégration d'APIs tierces (football-data.org, Google Gemini, Brevo) et sécurisation multicouche (JWT, bcrypt, rate limiting, transactions ACID). L'ensemble représente 24 routes API REST, 20 tables en base de données, et plusieurs modules spécialisés (calcul de points, synchronisation de scores, email, chatbot).

Les compétences acquises au cours de ce projet sont multiples : maîtrise de l'architecture client-serveur découplée, gestion des états asynchrones complexes, conception de schémas de base de données relationnels, sécurisation d'une API REST, développement mobile réactif avec Riverpod, et gestion de projet en autonomie avec respect d'un cahier des charges structuré.

Perspectives d'amélioration : plusieurs évolutions sont envisagées pour les versions futures. Premièrement, le déploiement en production sur une infrastructure cloud (AWS ou Railway) avec containerisation Docker et reverse proxy Nginx. Deuxièmement, l'activation du microservice Python de recommandations ML de joueurs basées sur les statistiques historiques. Troisièmement, l'intégration complète des notifications push Firebase (FCM pour Android, APNs pour iOS). Quatrièmement, l'ajout d'un mode spectateur permettant de suivre des équipes d'autres utilisateurs sans en être membre. Enfin, la création d'une interface web (Next.js) complémentaire à l'application mobile pour couvrir l'ensemble des plateformes.

WC Fantasy représente une démonstration concrète de notre capacité à concevoir et réaliser un système logiciel complet, de la spécification à la mise en œuvre, en appliquant les meilleures pratiques du développement web et mobile modernes.

Bibliographie et Annexes

Bibliographie et Webographie

[1] Mozilla Developer Network (MDN). *REST API — HTTP Methods*. https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods. Consulté en 2025.

[2] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/. Consulté en 2025.

[3] Redis Ltd. *Redis Documentation — Commands Reference*. https://redis.io/docs/. Consulté en 2025.

[4] Socket.io Team. *Socket.io Official Documentation v4*. https://socket.io/docs/v4/. Consulté en 2025.

[5] Flutter Team — Google. *Flutter Documentation*. https://docs.flutter.dev/. Consulté en 2025.

[6] Riverpod. *Riverpod State Management Documentation*. https://riverpod.dev/. Consulté en 2025.

[7] JSON Web Tokens. *JWT.io — Introduction to JSON Web Tokens*. https://jwt.io/introduction. Consulté en 2025.

[8] football-data.org. *Football Data API Documentation*. https://www.football-data.org/documentation/quickstart. Consulté en 2025.

[9] Google. *Gemini API Documentation*. https://ai.google.dev/docs. Consulté en 2025.

[10] Brevo. *SMTP Email API Documentation*. https://developers.brevo.com/. Consulté en 2025.

[11] OWASP Foundation. *OWASP Top 10 Web Application Security Risks*. https://owasp.org/Top10/. Consulté en 2025.

[12] Statista. *Fantasy Sports Market Size 2023–2030*. https://www.statista.com. 2024.

[13] FIFA. *FIFA World Cup 2026 Official Website*. https://www.fifa.com/worldcup/2026/. Consulté en 2025.

[14] node-cron. *node-cron Documentation*. https://github.com/node-cron/node-cron. Consulté en 2025.

[15] Nodemailer. *Official Nodemailer Documentation*. https://nodemailer.com/about/. Consulté en 2025.

[16] Node.js. *Node.js Official Documentation*. https://nodejs.org/en/docs/. Consulté en 2025.

[17] npm. *npm Documentation – Publishing packages*. https://docs.npmjs.com/. Consulté en 2025.

Annexes

Annexe A — Endpoints REST principaux

Méthode + Route	Description & Authentification
POST /api/auth/register	Créer un compte → email + username + password
POST /api/auth/login	Connexion → retourne access_token + refresh_token
POST /api/auth/refresh	Rafraîchir l'access token avec refresh_token
POST /api/auth/forgot-password	Demander un lien de réinitialisation par email
POST /api/auth/reset-password	Réinitialiser le mot de passe via token email
GET /api/players	Lister les joueurs (filtres : position, club, nationality) — Auth
GET /api/players/ :id	Détails d'un joueur — Auth
GET /api/fantasy-team/my-team	Récupérer son équipe — Auth
POST /api/fantasy-team/my-team/players	Ajouter un joueur à l'équipe — Auth
DELETE /api/fantasy-team/my-team/players/ :id	Retirer un joueur — Auth
GET /api/leagues	Mes ligues — Auth
POST /api/leagues	Créer une ligue — Auth
POST /api/leagues/join	Rejoindre via code invitation — Auth
GET /api/leagues/ :id/battles	Batailles d'une ligue — Auth
GET /api/battles/ :id	Détail d'une bataille — Auth
GET /api/gameweeks	Liste des journées de jeu — Auth
POST /api/gameweeks/ :id/scores	Saisir les scores d'une journée — Admin
POST /api/predictions	Soumettre une prédiction — Auth
GET /api/predictions/gameweek/ :id	Stats communautaires d'une journée — Auth
GET /api/news	Actualités football — Public (cachées Redis)
POST /api/chatbot/chat	Envoyer un message au chatbot Gemini — Auth
GET /api/photos/player/ :id	Photo d'un joueur (cascade DB→cache→Wikipedia→fallback)
POST /api/admin/photos/backfill	Enrichissement photo en masse — Admin
DELETE /api/transfers/leagues/ :id/transfer	Effectuer un transfert entrant/sortant — Auth

TABLE 4.6 – Principaux endpoints REST de l'API

Ce tableau recense les principaux endpoints REST exposés par l'API du projet, en précisant la méthode HTTP, la route, la description de l'action et le niveau d'authentification requis.

Annexe B — Variables d'environnement (.env)

Variable	Description
PORT	Port du serveur Express (défaut : 3000)
NODE_ENV	Environnement : development / production
JWT_SECRET	Clé secrète pour signer les access tokens
JWT_REFRESH_SECRET	Clé secrète pour signer les refresh tokens
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	Connexion PostgreSQL
REDIS_HOST / REDIS_PORT / REDIS_PASSWORD	Connexion Redis
BREVO_API_KEY	Clé API Brevo pour l'envoi d'emails SMTP
EMAIL_FROM	Adresse expéditeur des emails (ex : noreply@fantasyfc2026.com)
FOOTBALL_DATA_API_KEY	Clé API football-data.org pour la synchronisation des scores
GEMINI_API_KEY	Clé API Google Gemini pour le chatbot IA
CORS_ORIGIN	Origine(s) autorisées pour CORS (ex : http ://localhost :3001)
ADMIN_EMAIL / ADMIN_PASSWORD / ADMIN_USERNAME	Identifiants du compte administrateur initial
CLIENT_URL	URL de l'application cliente (pour les liens email de reset MDP)
APP_NAME	Nom de l'application affiché dans les emails

TABLE 4.7 – Variables d'environnement

Ce tableau liste toutes les variables d'environnement nécessaires au démarrage de l'application, incluant les paramètres de connexion aux bases de données, les clés API et les secrets d'authentification.

Annexe C — Barème complet des migrations SQL

Fichier	Contenu
001_initial_schema.sql	Création des 20 tables principales + indexes de base
002_add_player_photo.sql	Ajout de la colonne photo_url à la table players
003_add_admin.sql	Ajout de la colonne role à users (user / admin)
004_extended_user_profile.sql	Ajout avatar_url, bio, date_of_birth à users
005_national_teams.sql	Création des tables national_teams et national_fixtures
006_national_teams_nationality_key.sql	Ajout de la contrainte UNIQUE sur national_teams.nationality
007_budget_constraint.sql	Contrainte CHECK (budget ≥ 0) sur teams.budget
008_add_indexes.sql	9 index FK sur teams, battles, player_scores, refresh_tokens, predictions
009_add_saves_column.sql	Colonne saves INTEGER DEFAULT 0 sur player_scores
010_add_xp_column.sql	Colonne xp INTEGER DEFAULT 0 sur users (séparation XP / points fantasy)

TABLE 4.8 – Liste des migrations SQL

Ce tableau décrit les migrations SQL successives appliquées à la base de données PostgreSQL du projet, chacune correspondant à une évolution incré